Appendix — full feedback register, XLS-65 / XLS-66

This is the supporting evidence for `/FEEDBACK.pdf`, the three-page developer report at the repository root: the complete 40-finding register, with transaction hashes, file and line references, verbatim error strings, upstream issue and PR numbers, and the three findings we withdrew. Nothing here is rewritten for the report; the report selects from it.

De Vinci Blockchain — XRPL Lending Protocol Hackathon, 12–13 September 2026. Track 2 (closed-ended vault, Lending Protocol V1.1), flavour Loaded.

---

1. WHAT THIS IS, AND WHERE IT COMES FROM

We set out to build a vault-solvency oracle on the Lending Protocol. To do that we had to originate real loans on Devnet, service them, impair them, default them, liquidate first-loss cover, and read the result back correctly. We got all the way through: loans originated, payments made, `tfLoanImpair` and `tfLoanDefault` applied, cover liquidated, first-loss arithmetic reproduced to the drop, and a fragility score published as a native XLS-47 Oracle object that gates deposits through an XLS-70 credential and a Permissioned Domain. Every claim below carries a transaction hash, a file and line, a verbatim error string, or a registry timestamp.

This document is what it cost to get there.

We want to be clear about the register. The protocol design is good, and in one respect it is better than the incumbents it will be compared against: a closed-ended vault forbids maturity transformation at the ledger level. `VaultWithdraw` is refused for the entire Investment phase and `LoanSet` is refused for any loan maturing after `RedemptionDate`. Maple and Goldfinch both had to retrofit lockups and pro-rata redemption *after* a run; XRPL makes the run mechanism illegal in the primitive. That is a real design achievement and it is not what this document is about.

What this document is about is the distance between the protocol that is running on Devnet today and the protocol that a developer can read about, install a library for, and follow a tutorial into. That distance is currently large, and almost all of it is fixable in days rather than quarters — several items are a single line.

Disclosure, per the event terms. Reproduction transactions were executed against Devnet on 2026-09-10 and 2026-09-11 as pre-event reconnaissance. Every finding in this document was re-verified on 2026-09-12, during the event window, against the live network, against the current `master` of each repository, and against the open pull-request sets — including file-level diffs, not just titles. Where re-verification changed a finding, we say so in the finding itself. Three findings were withdrawn during reconnaissance and are listed in §10 rather than deleted, because a report you cannot check is worth nothing.

That re-verification changed this document materially, and §13 records exactly how. Two of our findings were fixed upstream during the event window itself. Five more turned out to have prior art we had missed on the first pass — including our own lead finding — and now cite it. One claim about SDK coverage was simply wrong and has been removed. We would rather hand you a shorter document that survives checking than a longer one that does not.

1.1 Environment

Network	XRPL **Devnet**, `wss://s.devnet.rippletest.net:51233` / `https://s.devnet.rippletest.net:51234`
Server (initial probing, 2026-09-10/11)	rippled **3.4.0-rc4**, `network_id` **2**
Server (all re-verification, 2026-09-12)	rippled **3.4.0-rc5**, `network_id` **2**, ledger 5,240,601 at 01:19:18 UTC
Amendments	89 enabled, including `SingleAssetVault`, `LendingProtocol`, `LendingProtocolV1_1`, `fixCleanup3_4_0`, `Credentials`, `PermissionedDomains`, `PriceOracle`, `ConfidentialTransfer`. `LendingProtocolV1_2` and `fixCleanup3_5_0` are not present.
Clio	`https://clio.devnet.rippletest.net:51234`
Node	v24.13.0
JS libraries (baseline for every re-test)	`xrpl` **5.2.0** stable with `ripple-binary-codec` **2.11.0**. Findings were originally hit on `xrpl` 5.1.0 / rbc 2.10.0, and cross-checked against 5.2.0-beta.0, 5.2.0-beta.1 and rbc 2.11.0-beta.1. Also `ripple-keypairs`, `decimal.js` 10.6.0
TypeScript	5.6.3, `--strict`
Python	3.13.2; `xrpl-py` **5.1.0** (still PyPI `latest` on 2026-09-12) and **5.2.0b0** (prerelease), both installed from PyPI rather than read from the repository

1.2 The release timeline matters, so here it is

All timestamps are UTC, taken from the npm and PyPI registry `time` maps on 2026-09-12.

PUBLISHED (UTC)	PACKAGE
2026-08-24 21:07	`ripple-binary-codec` 2.10.0
2026-08-25 00:29	`xrpl` **5.1.0** — npm `latest` for the next 17 days
2026-09-10 02:19	`ripple-binary-codec` 2.11.0-beta.0
2026-09-10 13:42	`xrpl` 5.2.0-beta.0 (`beta-experimental`)
2026-09-11 00:00	`xrpl-py` **5.2.0b0** (prerelease; PyPI `latest` is still 5.1.0)
2026-09-11 16:26	`ripple-binary-codec` 2.11.0-beta.1
2026-09-11 16:59	`xrpl` 5.2.0-beta.1
2026-09-11 20:56	`ripple-binary-codec` **2.11.0**
2026-09-11 22:20	`xrpl` **5.2.0 — 8 h 40 m before this hackathon opened** (00:20 Paris)

For the 17 days from 2026-08-25 to 2026-09-11, `npm install xrpl` produced a library that could not serialize the first transaction of the Lending Protocol on the only network where the Lending Protocol runs. That window closed the night before the event. We think the pattern — not the individual bug — is the finding.

2. SEVERITY RUBRIC

We use three levels. The rubric is stated before it is used so that you can disagree with a rating without having to re-derive it.

P0	**Blocks a developer entirely.** No supported path exists in the published tooling, or the documented path cannot succeed. The cost is a lost day or an abandoned project, and the developer has no way to know that the fault is not theirs.
P1	**Costs hours and has a non-obvious workaround.** The platform works, but only after you have read C++ transactor source, diffed a package tarball, or submitted a transaction to find out what a rule is. Silent wrong-value bugs live here regardless of how small the fix is, because the cost is not the fix, it is the hours before you know you have a problem.
P2	**Friction, or a documentation inaccuracy.** You lose minutes, or you are told something that is not true and you find out cheaply.

Every finding also carries a **status**, which is a claim about the world and not about severity:

NEW	We searched and found nothing upstream. The searches we ran are listed in §11.2.
ALREADY REPORTED	An open upstream issue covers it. We cite the number and add the lending-specific reproduction as a comment rather than opening a duplicate.
PREVIOUSLY RAISED, CLOSED UNTRIAGED	Someone reported it, no maintainer responded, and it was closed without a resolution. We cite it, say what happened to it, and say why we are raising it again.
SPECIFIED BUT UNMERGED	A correct specification or fix exists as an open pull request and has not landed.
PARTIALLY TRACKED	An issue or PR exists but is a placeholder, or covers only part of the problem.
FIXED DURING OUR WINDOW	It was live when we hit it and is fixed in a release published since. Reported anyway: the fix landed hours before the event, and the gap between released and documented is itself the finding.
WITHDRAWN	We reported it internally, re-verified it against `master` today, and it does not reproduce. §10.

3. INDEX OF FINDINGS

40 findings, plus three we withdrew (§10). Statuses are as of 2026-09-12 and were re-checked against the live trackers that morning; §13 lists what moved.

ID	SEV	STATUS	COMPONENT	FINDING
D1	**P0**	**Specified but unmerged / partially tracked**	XLS-65, XLS-66, docs	`LendingProtocolV1_1` ships behaviour whose specification has been an open PR for 53–80 days, and one rule that no *standard* states
X1	**P0**	**Previously raised, closed untriaged** (`rippled#6487`)	rippled metadata	The first impairment of a healthy vault emits `PreviousFields: {}` and is invisible to every metadata-diffing indexer
D2	P0	NEW	xrpl-dev-portal	The lending tutorial chain does not complete, and dies as a `TypeError` instead of reporting a result code
L1	P0	Fixed during our window	`ripple-binary-codec` ≤2.10.0	`npm install xrpl` could not serialize a close-ended `VaultCreate`; `validate()` and `autofill()` both passed it
L2	P0	NEW	xrpl-py (all versions)	No published version can sign a `LoanSet` counterparty signature; the `0x43505400` prefix does not exist in the package
L3	P0	NEW	xrpl-py 5.1.0	`pip install xrpl-py` cannot construct a close-ended `VaultCreate`
M1	P0	NEW (related: `XRPL-Standards#611`)	rippled API	No state proof on any read API, so nothing off-ledger can verify a fact about XRPL ledger state
L4	P1	NEW	xrpl.js, all versions	`LoanSet` missing from `txToFlag`; the error names `tfInnerBatchTxn` as the only valid flag; and object-form `Flags` never reach the signer
L5	P1	Already reported (`xrpl-py#1016`)	xrpl-py	Dict-form `Flags` fail open to `0`, including the library's own documented spelling, on a lending transaction
L6	P1	NEW	both libraries	The typed `ledger_entry` request model cannot select a `Loan` or a `LoanBroker`
L7	P1	NEW	xrpl.js 5.2.0	`VaultInfoResponse` omits the four fields that define a close-ended vault
U1	P1	NEW	rippled	`tecNO_PERMISSION` means four different things on the lending path
U2	P1	NEW	rippled	`tecINSUFFICIENT_FUNDS` cannot distinguish "you do not own that" from "the vault has no cash"; no partial fill
U3	P1	Partially tracked (`XRPL-Standards#625`)	XLS-66	Seven rate fields are in 1e-5 units; the only written-down source is a constant inside `node_modules`
U4	P1	NEW (the layer contradiction); `Scale` range already reported as `xrpl.js#3435`	rippled + xrpl.js	Fields that may not be set to their default value, in a client that requires them
U5	P1	Already reported (`xrpl-dev-portal#3612`), with a new ask	rippled API	Absent means zero across every read surface, with no normalising option
M2	P1	NEW	rippled API	No native read of what a vault share is worth, and one valuation rule no external reader can reproduce
M3	P1	NEW	rippled / Clio	No enumeration and no event stream for vaults, brokers or loans
D3	P1	Partially tracked (`xrpl-dev-portal#3923`)	XLS-65, docs	Three `tec` outcomes that gate the lifecycle are absent from XLS-65, and the docs fix for them is unmerged
D4	P1	NEW	xrpl-dev-portal	The faucets list still advertises a Lending Devnet whose hostnames do not resolve
D5	P1	NEW	XLS-65 §3.9.1	`vault_info`'s documented request parameter name is wrong
D6	P1	NEW	XLS-65 §4.2	`vault_list` is fully specified with three worked examples and unimplemented on both rippled and Clio
D7	P1	NEW	xrpl-dev-portal	The canonical ledger-entry short-names table omits `Vault`, `Loan` and `LoanBroker`
D9	**P1**	**NEW — settled by experiment** (was P2)	`XRPL-Standards#587` §7	The draft and the implementation disagree about the loan-maturity buffer, and the draft is the one that is wrong
L8	P2	Already reported (`xrpl-py#978`)	xrpl-py	`get_amount_value()` returns `float` in a library whose Number fields carry 19 significant digits
L9	P2	Already reported (`xrpl.js#3154`)	xrpl.js	`DEFAULT` fields typed as required; a defaulted loan deletes four of them
L10	P2	NEW	xrpl.js 5.2.0	`Loan.OverpaymentFee` typed as a string in the ledger model; the wire type is `UInt32`
L11	P2	Specified but unmerged (`xrpl.js#3229`, `#2683`, `#2597`, `xrpl-py#899`)	both libraries	No custom-definitions API on the high-level client, which is what you need precisely when the release lags the network
L12	P2	NEW	xrpl.js 5.2.0	The library writes to the console on the success path, and the vendor's own tutorial monkey-patches it away
L13	P2	Specified but unmerged (`xrpl.js#3430`)	xrpl.js	`VaultCreate` autofill charges the owner reserve and bypasses `maxFeeXRP`

ID	SEV	STATUS	COMPONENT	FINDING
U6	P2	NEW	xrpl.js	A `tem`/`tel` rejection throws an error carrying no structured result; three failure layers surface three different ways
U7	P2	NEW	xrpl.js	Client-side validators disagree with the server and cannot be bypassed through the supported API
U8	P2	NEW	rippled	The protocol's minimum feedback loop is about three minutes and there is no way to shorten it on a test network
U9	P2	NEW	docs	Phase gates evaluate against ledger `close_time`, which lags wall clock; nothing says so
U10	P2	NEW; the `VaultID` sub-case is `XRPL-Standards#497` + `rippled#6528`	XLS-66 / xrpl.js	One transaction, three field-shape conventions, and two write-once fields behind an undifferentiated error
M4	P2	Partially tracked (`XRPL-Standards#623`, closed unmerged)	XLS-66, rippled API	XLS-66's two statements of the cover-liquidation formula disagree, and no read API previews it
D8	P2	NEW	XLS-66 §3.11.1	`LoanPay` is given transaction type 83; it is 84
D10	P2	Partially tracked (`XRPL-Standards#616`, `#555`)	XLS-65, XLS-66	Twelve Invariants sections, plus XLS-65's Rationale and Security Considerations, are `TBD`
D11	**P1**	**NEW**	XLS-65, XLS-66, docs	Both standards state the arithmetic completely and evaluate none of it. No number appears anywhere, and a rule nobody has evaluated reads as a rule with no consequences
M5	**P1**	**NEW — found while building**	rippled	A LoanBroker's action history cannot be reconstructed by filtering: `LoanManage` names no broker and an impairment does not touch the broker object, so impairments vanish and the consumer cannot tell
W1–W3	—	**Withdrawn**	—	See §10

4. LEAD FINDING D1 — `LENDINGPROTOCOLV1_1` SHIPS BEHAVIOUR WHOSE SPECIFICATION IS UNMERGED, AND ONE RULE THAT NO STANDARD STATES

Severity: P0. Status: specified but unmerged (D1.a, D1.b); documented downstream but unspecified (D1.c). Category: documentation — but it is the root cause of most of §9 and half of §6.

What a developer expects

`LendingProtocolV1_1` is enabled on Devnet. XLS-65 has an "Amendments" section. A developer reads it to find out what the amendment changes.

What actually happens

XLS-65 §2.8, fetched from `master` on 2026-09-12, in full:

```
- `LendingProtocolV1_1`, as described in [XLS-65.1](./65.1/README.md):
  - adds an optional `MemoData` field to `VaultDelete` that, if present, must be 1–256 bytes.
```

The referenced XLS-65.1 is an 83-line, 4,385-byte document titled "**Vault Deletion Memo**".

Grep counts across the published `XLS-0065-single-asset-vault/README.md` (76,942 bytes), `XLS-0065-single-asset-vault/65.1/README.md` and `XLS-0066-lending-protocol/README.md` (163,018 bytes), all on `master`, 2026-09-12:

TERM	OCCURRENCES
`VaultKind`	**0**
`SubscriptionDate`	**0**
`RedemptionDate`	**0**
`LEVersion`	**0**
"close-ended" / "closed-ended"	**0**
"cash basis"	**0**

In reality `LendingProtocolV1_1` does at least three things beyond the `MemoData` field, and all three change what a developer must build:

D1.a — Close-ended vaults exist, and the spec for them is PR #587, open for 53 days

`XRPLF/XRPL-Standards` **PR #587 "Closed-ended Vault"** — opened 2026-07-21 by `a1q123456`, last updated 2026-09-08, +495 lines, not a draft, **unmerged**. It is authored by the same three people as XLS-65 (Jingchen Wu, Vita Tumas, Gregory Tsipenyuk). It mentions `VaultKind` on 32 lines, `SubscriptionDate` on 39 and `RedemptionDate` on 48 (33, 47 and 58 occurrences). It specifies the phase model and the `LoanSet` phase gate, and both match what we observed on the wire. Its `MIN_INVESTMENT_PERIOD` does not: §2.4 gives 60 seconds, and Devnet brackets at 180 (179 -> `temMALFORMED`, 180 -> `tesSUCCESS`, `0C6127F4E9042C5EB1DD85A7274FC0748F19D9D1839256459ED5D206D9A14D06`). Raised on the PR itself.

It is simply not published. `xrpl.js` PR #3456 and `xrpl-py` PR #1034 — the library support for this feature, both also unmerged — cite it as their source. So the library maintainers can read it and application developers cannot.

D1.b — Devnet vaults are cash-basis, and the spec for that is PR #582, open for 58 days

Under `LEVersion: 1`, loan origination adds **nothing** to `Vault.AssetsTotal`. It only lowers `AssetsAvailable` and raises `LoanBroker.DebtTotal`. Interest reaches `AssetsTotal` only as it is actually paid. The published XLS-66 describes up-front accrual instead, so anyone who implements the published accounting model computes the wrong NAV.

Measured, live:

Origination of 10 XRP against a 50 XRP vault	`AssetsTotal` **50000000 → 50000000** (unchanged, and absent from `PreviousFields`), `AssetsAvailable` 50000000 → 40000000, `DebtTotal` 0 → 10000000
Evidence	`CF8E3BDD7BC9902EBA2D85E7674A8E0E7C5B7D31644EE543017E6CAD1142C890`, and independently `DEFF4874E37133CFD5AD4C4201CCC6E1703FA500B245E8F9164F4BDDB99A28DC`
One on-time `LoanPay` of 1,666,674 drops	`AssetsTotal` 51000000 → **51000011** (+11 drops = the interest slice only); `AssetsAvailable` +1,666,674 (the whole payment)
Evidence	`5814A9CF75895EF326A042795FBBAA5D9B8AAA5C686E60B482D754442B371C0B`
Invariant that held at ~30 observations	`AssetsTotal == AssetsAvailable + DebtTotal`
Re-measured on rippled **3.4.0-rc5**, 2026-09-12	origination of a 10 XRP loan against a 40 XRP vault left `AssetsTotal` at 40000000 and absent from `PreviousFields`, moved `AssetsAvailable` to 30000000, and preserved `AssetsTotal == AssetsAvailable + DebtTotal` (`F993EB6BA7CCBE3B2461148DD77DFAE979A0A902AB937BB9D390050539BA7D37`)

`XRPLF/XRPL-Standards` **PR #582 "XLS-65 / XLS-66: Principal-only Vault/LoanBroker accounting under LendingProtocolV1_1"** — opened 2026-07-16 by `Tapanito`, updated 2026-09-11 21:07 UTC, +464/−135 across six files including a new `XLS-0065-single-asset-vault/65.1/vault-cash-basis.md`. `LEVersion` appears 70 times across 57 added lines. **Unmerged.** It also restructures 65.1 into `vault-cash-basis.md` + `vault-memo.md`, which is why the published 65.1 reads as if the whole amendment were a memo field: that framing is an artefact of an unmerged restructure.

(`PR #570 "XLS-65/66: Add LendingProtocolV1_1 two-step loan creation"`, opened 2026-06-24 by `Tapanito`, +375/−155, is the third: 80 days open.)

D1.c — `LoanBrokerSet` requires a close-ended vault, and no *standard* says so

This is the rule that stops every new developer. It is enforced in merged code, stated in a merged rippled PR, and written up in an open docs PR — and it is absent from every specification artefact, published or in flight. We had this filed as "undocumented everywhere" until we read the open pull requests file by file on the morning of 12 September; the corrected version is narrower and, we think, more useful.

We proved the behaviour by exhaustion, isolating the vault-kind requirement from the documented owner requirement:

SUBMITTER	VAULT KIND	RESULT	HASH
non-owner	open-ended	`tecNO_PERMISSION`	`A260515DE5C615E1A1A2E299D98FA6D652ADD75F3A2C105B7497965323FBF226`
non-owner	close-ended	`tecNO_PERMISSION`	`FCF6DC50385AC1D143EC25BB5204B040A607E64EA9BCCE7D78D4F2BC5E77E70B`
vault owner	**open-ended**	`tecNO_PERMISSION`	`86AC8182A100EEDA32495706C06CDAC6D522B0E0772F5F65696064B6F3C0E5ED`
vault owner	**close-ended**	`tesSUCCESS`	`CFC576DF9DCF7DB12059C559F93BE6F6094011820796859A1E814586C0E4AD5F`

Row 3 is the one that matters: the submitter *is* the vault owner, so the documented rule is satisfied, and it still fails. Reproduced independently on four separate vaults across three of our probes (`F4646C83891785A87881DBBCB83D0B0F2F64A53FA1DD169D1E80F3185AEC9DA8`, `76E384CC9FE14628B7DE9E8739A970B05AE04B09FC29EE76B887AFF2EF211418`, `45A3264CD4B02B41458B38468DAA03BE889171764CCC9A85416D96C1DD2934CA`, `D6A3C93E0389AA705B96C59EA395228796A31DF08429C30102A5520AE6FB2306`), including with the transaction stripped to `Account` +

`VaultID` to eliminate every optional field as a cause (`F4A97E9BD627DB7397AD33866C2830D286195863CDAF9EDBBC4DBE4CC696417B`). Re-proved on rippled 3.4.0-rc5 on 2026-09-12: owner + open-ended vault → `tecNO_PERMISSION` (`9049D56423AB5E61B0D1FF157666DA6CD9DF0D504333B988EF27753597BB22D5`).

The rule is written down in C++. `LoanBrokerSet.cpp`, preclaim, the create-a-new-broker branch — lines **149–161** at commit `421af6db796631da4ff8b78f5d3bafae0fd3ca32`, the comment and the check together:

```
// LP V1.1: only closed-ended vaults may host a loan broker. The
// lending protocol relies on the closed-ended Subscription /
// Investment / Redemption phase structure; attaching a broker to
// an open-ended vault has no well-defined lifecycle. VaultCreate
// stays unrestricted so existing open-ended flows keep working;
// the constraint is enforced here, at the point where the vault
// is first bound to the lending protocol.
if (ctx.view.rules().enabled(featureLendingProtocolV1_1) &&
    getVaultKind(sleVault) != VaultKind::ClosedEnded)
{
    JLOG(ctx.j.warn()) << "LoanBroker requires a closed-ended Vault.";
    return tecNO_PERMISSION;
}
```

That comment is well written and clearly intended for exactly the confusion it causes. It should be in the spec.

Where it is written down. `XRPLF/rippled#8076` "fix: Reject open-ended vaults at `LoanBrokerSet`" (merged 2026-08-26) introduced that C++ and states the rule and its rationale in its description, with a before/after behaviour table. `XRPLF/xrpl-dev-portal#3923` (open, see the end of this section) adds it to `loanbrokerset.md` both as an admonition and as a `tecNO_PERMISSION` table row. Both are correct, and #3923 carries a nuance we had missed and could not have derived from our own probes: the constraint binds only brokers **created after** the amendment, so a broker attached to an open-ended vault beforehand keeps originating loans. We are glad to be able to correct our own draft with it.

Where it is not is the standard. XLS-66 **§3.3.3.2 Protocol-Level Failures** enumerates ten numbered failure conditions for `LoanBrokerSet` — five for the create branch, four for the modify branch, one for precision. Verified verbatim on `master` today, and verified again on PR #582's branch (updated yesterday): the close-ended requirement is not among them in either.

And PR #582 explicitly defers it. Line 124 of its `XLS-0066-lending-protocol/README.md`:

```
- Closed-ended Vault phase checks on `LoanSet` and the closed-ended requirement on
  `LoanBrokerSet` are specified in [PR #587](https://github.com/XRPLF/XRPL-Standards/pull/587),
  not in this patch.
```

PR #587 contains the string `LoanBrokerSet` **zero times**. It mentions `LoanBroker` twice, both about owner-count blocking `LoanBrokerDelete`. **The two in-flight specification PRs point at each other, and neither of them specifies the rule.** Discussion #590 does not either. *(True at the event. Corrected 2026-10-13: #587 was merged on 2026-09-15 with the restriction, after our comment; since #636 it is §3.3 of* `66.1.2-closed-ended-loan-gates.md`*.)*

So the ask is narrower than we first wrote it, and it is cheap: the behaviour is agreed, implemented and already documented downstream. Please land it in the standard, because XLS-66 §3.3.3.2 is where an implementer looks for `LoanBrokerSet`'s failure conditions, and it lists ten of them without this one.

Why this is P0 rather than a documentation nit

`VaultKind` is immutable. `VaultSet`'s transaction format does not admit `VaultKind`, `SubscriptionDate` or `RedemptionDate` at all — rippled rejects the blob at deserialization, before preflight, with `invalidTransaction: Field 'VaultKind' found in disallowed location.` (five variants tested; the `{Data}`-only control succeeded, `284A6C26CD5ED2B23A32663D6201F9416131074F125DEF3F29D6E61A20A3EB67`).

So a developer who follows the published docs builds an open-ended vault, issues its share MPT, takes LP deposits into it, and only then discovers there is no upgrade path and no conversion. The vault, the issuance and every LP position have to be torn down and rebuilt. The cost of the missing paragraph is not minutes of reading, it is a rebuild.

Suggested fix

1. **Merge PR #587 and PR #582**, or land them as published drafts, and **merge** `xrpl-dev-portal#3923`

(see the end of this section). They are correct. The amendment they describe is enabled on a public network that a Ripple-sponsored hackathon is being run on.

1. **Add an 11th protocol-level failure condition to XLS-66 §3.3.3.2**, in the "creating new" branch:

"`Vault(VaultID).VaultKind` *is not* `ClosedEnded`*. (*`tecNO_PERMISSION`*) — requires* `LendingProtocolV1_1`*."* This is one line and it is the single highest-value line in this document.

1. Give `LoanBrokerSet` a distinct result code for this case, or emit the transactor's JLOG reason in the

metadata. Three of four failing combinations returning the same undifferentiated `tecNO_PERMISSION` is what turns a five-minute problem into a two-hour one. See U1 — and note that `XRPLF/rippled#7848` already prototypes exactly this, so this ask is support for work in flight rather than a new request.

1. Update XLS-65 §2.8 so that it describes the amendment rather than one field of it.

Already in flight, and it deserves to be said plainly

`XRPLF/xrpl-dev-portal#3923` "**Migrate LendingProtocolV1_1 docs from opensource**" is an **open pull request** — not an issue — by `oeggert`, opened 2026-09-11 19:56 UTC, base branch `release-3.4.0`, not a draft. Its body is empty, which is why it reads as a placeholder in a list view. Its diff is not: **18 files, +244/−97** when we read it at 2026-09-12 08:10 UTC, and it had been pushed to at 07:56 that morning, so those numbers move. We had it recorded as an empty-bodied placeholder, and that was our error; reading it properly changed three of our findings.

What it already lands, verified from the diff rather than the title:

- `loanbrokerset.md` — the closed-ended admonition and the `tecNO_PERMISSION` row, **which is D1.c**, plus

the pre-amendment carve-out we did not have.

- `vaultcreate.md` — `VaultKind`, `SubscriptionDate` and `RedemptionDate` field rows, the

`[180, 946708560)` bound, `tecEXPIRED`, and expanded `temMALFORMED` / `temDISABLED` conditions.

- `single-asset-vaults.md` — a "Closed-Ended vs Open-Ended Vaults" section with the three phases, and

cash-basis accounting: **D1.a and D1.b on the developer-facing side**.

- `vault.md`, `vault_info.md`, `loanbroker.md` — the new fields on the read surfaces (L7's server side).
- `loanset.md` and `vaultwithdraw.md` — three of the lifecycle `tec` outcomes in D3, and the 60-second

maturity buffer in D9, which we had been unable to source from any document.

This is good work and it closes the developer-facing half of D1. Two things remain, and they are the two we would now ask for:

1. **It is unmerged and it targets `release-3.4.0`, so none of it is on xrpl.org today.** We confirmed

`loanbrokerset.md` on `master` contains neither "closed-ended" nor `VaultKind`. Every participant at this weekend's hackathon is reading the old pages. Our contribution here is independent confirmation on the wire against rippled 3.4.0-rc5 that every rule in that diff behaves as written, and a request to prioritise the merge.

1. **It is documentation, not specification.** XLS-65 §2.8 still describes `LendingProtocolV1_1` as a memo

field, and XLS-66 §3.3.3.2 still enumerates ten `LoanBrokerSet` failure conditions without the closed-ended one. Merging #3923 fixes a developer's first hour. Merging #587 and #582 and adding one line to §3.3.3.2 fixes the record.

---

5. LEAD FINDING X1 — THE FIRST IMPAIRMENT OF A HEALTHY VAULT IS INVISIBLE TO METADATA DIFFING

Severity: P0. Status: PREVIOUSLY RAISED, CLOSED UNTRIAGED (`XRPLF/rippled#6487`). Category: missing primitive / observability. This is our own headline finding and it is the reason our prototype exists — and the prior report we found on re-verification is the best evidence in it.

What a developer expects

A vault's `LossUnrealized` goes from 0 to 10,000,000 drops. The transaction metadata records the change, so an indexer that diffs `PreviousFields` against `FinalFields` — which is the standard way to build one — sees it.

What actually happens

`PreviousFields` is an **empty object**.

Verbatim capture, `LoanManage` with `tfLoanImpair` (Flags 131072) on Devnet, transaction `8B9657E31D9130FE557682FE33E779789243EACEB0E0D97D83F5D6A3F477A1D5`, Vault ledger index `6717B5115871C2C1A5552C62B68F52A55C9C58086BD1FDC78D7DCE43ACDD773C`:

```
{
  "ModifiedNode": {
    "FinalFields": {
      "Account": "rhCcvBkBsg9Wm5kXSndxpcCcEQB6GZtvah",
      "Asset": { "currency": "XRP" },
      "AssetsAvailable": "41000000",
      "AssetsTotal": "51000000",
      "LEVersion": 1,
      "LossUnrealized": "10000000",
      "Owner": "rhQUhoAdLNCizws6NP8WD88o5Ks4ppJ5F3",
      "RedemptionDate": 842552594,
      "ShareMPTID": "0000000128362A77C7F5D3FD8BC2E9F7E6936E5B9004FC42",
      "SubscriptionDate": 842466494,
      "VaultKind": 1,
      "WithdrawalPolicy": 1
    },
    "LedgerEntryType": "Vault",
    "LedgerIndex": "6717B5115871C2C1A5552C62B68F52A55C9C58086BD1FDC78D7DCE43ACDD773C",
    "PreviousFields": {},
    "PreviousTxnID": "7C00D94F373DB05D9D895FF66EA68F5143867C9E8BBCC25FCEF816340534E795",
    "PreviousTxnLgrSeq": 5233047
  }
}
```


The vault is still on Devnet. `{"method":"vault_info","params":[{"vault_id":"6717B511…ACDD773C"}]}` returned it live at 2026-09-12 03:20 UTC.

Unchanged on rippled **3.4.0-rc5**: a fresh impairment submitted on 2026-09-12, `BF3C131BFCDB0BA867DDA4A3AC2F99360A7E4D7AA196F35BEB0906CA4FF9A94C`, returned `PreviousFields: {}` with `FinalFields.LossUnrealized` `"10000000"` and `AssetsTotal` unmoved at `40000000`.

Why

Corrected 2026-10-13: the mechanism is presence, not value. The submitted text said in places (the suggested fix below, U5 and §12) that rippled omits a field whose previous value was the type default; those passages are restated. The claim that the same absence hides the first `DebtTotal`, `CoverAvailable` *and* `AssetsTotal` *is kept, restated by presence and now cited with its transactions. Removed: our reading of the prior reporter's reasoning, and a heading that called the prior art our strongest evidence. See* `XRPLF/xrpl-dev-portal#3926`.

`PreviousFields` records the prior value of fields that were **present** on the object and changed. An optional field such as `LossUnrealized` (declared `SoeDefault`) is not serialised onto the object while it holds its type default, so it has no prior value to record. `LossUnrealized`'s type default is 0, so on a healthy vault the field is simply not there. (The rule is presence, not value: the same transaction records the Loan's previous `Flags` of `0`, because `Flags` *was* on the object.) And under cash-basis accounting (D1.b) impairment changes *nothing else* on the Vault node — `AssetsTotal` and `AssetsAvailable` are untouched. The net result is a `ModifiedNode` that an indexer reads as **touched but unchanged**.

What it costs

Naive NAV, `AssetsTotal / shares.OutstandingAmount`	51000000 / 51000000 = **1.000000**
Correct NAV, `(AssetsTotal − LossUnrealized) / shares.OutstandingAmount`	(51000000 − 10000000) / 51000000 = **0.803922**
Divergence	**1,961 bps**

A metadata-diffing indexer reports no change across a 19.61% fall in net asset value, on the single most important credit event the protocol has. Across a sequence of impairments we measured the exchange rate going 1.0 → 0.803922 → 0.631678 while `AssetsTotal` never moved once.

Why it survives casual testing — it is asymmetric

This is the part that makes it dangerous. Three cases, same field, three different outcomes:

EVENT	PREVIOUS VALUE	PREVIOUSFIELDS	VISIBLE TO A DIFF?
First impairment of a healthy vault	none (field absent, read as 0)	`{}`	**No**
`tfLoanUnimpair`	10000000	`{"LossUnrealized":"10000000"}`	Yes
Second impairment while one is outstanding	2000000	`{"LossUnrealized":"2000000"}` → final 5000000	Yes

Un-impairment evidence: `32454E10D0B2BAA13367AD7AC06EFC9B9E3A5D85C2C2F7BD740575BEB0B746CA`. Accumulation evidence: `968D2C735E3214B50073E0BCB6E682B4260CACFF3E3E2D54AA94E033FF09385F`.

So an indexer author who tests the reversal, or who tests a second impairment, sees correct metadata and concludes the system works. The only case that breaks is the transition from healthy to distressed — the one that matters.

It is also not specific to `LossUnrealized`. The same absence hides the first `AssetsTotal` of a vault (first `VaultDeposit`, `202F9600CC1D85266FFB27B70F26CE09AB0FE0423D4A8E77FA12F7F8D515F6C1`), the first `CoverAvailable` of a broker (first `LoanBrokerCoverDeposit`, `13ECAC8451FB861723953DFE8BA9EC8B642A14FA5255943E7C7B1349EB1DD562`) and its first `DebtTotal` (first `LoanSet`, `525FC003CF1D4C99B82E876DE945000A681230D4312C03DE1120EA9F905118AB`): in each, the node's `PreviousFields` has no entry for the field the transaction creates. `LossUnrealized` is just where it hurts most.

There is a second, related trap on the other side of the lifecycle: on `tfLoanDefault` rippled **deletes** `PrincipalOutstanding`, `TotalValueOutstanding`, `PaymentRemaining` and `NextPaymentDueDate` from the `Loan` object. `loan.PrincipalOutstanding` after a default is `undefined`, not `"0"`. This crashed our own indexer once before we guarded it.

Suggested fix, cheapest first

1. **Document it.** One paragraph on the transaction-metadata page: *"`PreviousFields` records a previous

value only for a field that was present on the object before the transaction. A field the transaction creates has no entry, so a `ModifiedNode` may appear with an empty `PreviousFields` while `FinalFields` carries a new non-zero value. Do not infer 'unchanged' from an empty `PreviousFields`."* Cross-link it from the `Vault` ledger-entry page and from `LoanManage`. This costs nothing and would have saved us a day.

1. **Emit the default.** When a field is created on an existing node, record its default value in

`PreviousFields`. This is the correct fix and it makes metadata self-describing, but it changes the metadata format and so needs an amendment.

1. **Short of either**, have `LoanManage` write the delta into its own transaction metadata, so the credit

event is legible without re-reading state.

What we shipped instead

Never diff `PreviousFields`. Re-read the `Vault` SLE with `vault_info` after every `LoanManage`, coalesce absent to `"0"`, and parse with `decimal.js` at precision 40. Our demo is a two-indexer race on this exact transaction: the naive indexer reports 1.0000, the correct one reports 0.8039, and both are reading the same ledger.

Prior art

Our first draft of this finding said "prior art: none found". That was wrong, and the way it was wrong matters, so here is the whole of it.

`XRPLF/rippled#6487` "**LoanManage - Vault Object PreviousFields empty (Version: 3.1.1)**", opened **2026-03-05** by `zgrguric` — GitHub company `@XRPLWin`, the operator of the XRPLWin explorer — reports exactly this. In his words: *"Transaction result of type* `LoanManage` *with flag* `tfLoanImpair` *… has modified Vault object, Vault object correctly gained new* `LossUnrealized` *field, but PreviousFields are empty."* He attached a live Devnet reproduction. **He closed it himself the same day, with a single comment: "Not a bug."** No maintainer replied. Nothing was documented as a result, and the behaviour is unchanged six months later on 3.4.0-rc5.

We think that closure *is* the finding. The operator of an XRPL explorer — precisely the metadata-diffing indexer author this trap is built for — hit it on Devnet and closed it as not a bug. rippled is behaving as designed. What is missing is the one paragraph that says so, and says that "touched but unchanged" is not a safe inference. Nobody triaged it, so nobody wrote that paragraph.

We are re-opening the same ground with what #6487 did not have: the NAV arithmetic (1,961 bps), the asymmetry table above that shows why the bug survives casual testing, and a request that the behaviour be documented rather than closed. We will comment on #6487 with our reproduction rather than open a duplicate.

On our own process: §11.2 claims we searched `LossUnrealized` across `XRPLF` before marking anything NEW. Re-running that search on 2026-09-12 returns #6487 as its first result. We missed it, on our lead finding, and we would rather say so here than have it found for us.

---

6. CLIENT LIBRARY GAPS

L1 — `npm install xrpl` could not serialize the protocol's first transaction, and client-side validation passed it

P0 · `ripple-binary-codec` ≤ 2.10.0, as shipped by `xrpl` ≤ 5.1.0 · FIXED DURING OUR WINDOW (`ripple-binary-codec` 2.11.0, 2026-09-11 20:56 UTC), with two residual P1s

Expected: the tutorials say `npm install xrpl`. That library builds a vault.

Actual: for the 17 days from 2026-08-25 to 2026-09-11, npm `latest` was 5.1.0, whose bundled `ripple-binary-codec` 2.10.0 had none of `VaultKind`, `SubscriptionDate`, `RedemptionDate` or `LEVersion`.

```
// npm install xrpl@5.1.0
const sub = Math.floor(Date.now()/1000) - 946684800 + 3600
const tx = { TransactionType:'VaultCreate', Account: wallet.classicAddress, Asset:{currency:'XRP'},
             VaultKind:1, SubscriptionDate:sub, RedemptionDate:sub+3600, WithdrawalPolicy:1 }

validate(structuredClone(tx))            // PASSES
await client.autofill(structuredClone(tx))  // PASSES
wallet.sign(await client.autofill(tx))   // Error: Field VaultKind is not defined in the definitions
```

Fixed — and the credit belongs to the codec, not to the `xrpl` major. The serialization half of this was fixed in `ripple-binary-codec` **2.11.0** (published 2026-09-11 20:56 UTC). Because the codec range floats, that fix reaches *any* fresh install: we confirmed on 2026-09-12 that `xrpl@4.6.0` and `xrpl@5.2.0` both resolve `ripple-binary-codec@2.11.0`, and both now encode `VaultKind` (UInt8 nth 22), `SubscriptionDate` (UInt32 nth 75), `RedemptionDate` (UInt32 nth 76) and `LEVersion` (UInt8 nth 6). `LoanPay` is 84. The definitions hash moved to `0F89957938A9185335A2ACD799EDDF3965F349E2E482A0CDD97094A1E4DB9FE7`, which incidentally fixes a stale-hash problem we had noted across 2.10.0 and 2.11.0-beta.0. End to end: a plain `Wallet.sign()` + submit of a close-ended `VaultCreate` returned `tesSUCCESS` on rc5, `82552E13DBE3B793E831B61A2A98E092545EB015306C13236D8EDF84B6EFAC19`.

What `xrpl@5.2.0` genuinely adds is the *signing* half, which is L2: 4.6.0's `Wallet/utils.js` `computeSignature(tx, privateKey, signAs)` takes no signing role and always calls `encodeForSigning` (prefix `0x53545800`), and 5.2.0 introduces the `SIGNING_ENCODERS` table. We separate the two because the distinction changes the advice: a team pinned to `xrpl@4.x` can build a vault today but still cannot counterparty-sign a loan.

Residual 1, still P1 and still NEW: the *failure mode* is unchanged. `validate()` silently ignores unknown fields, so a codec gap surfaces as an opaque binary-codec error at `Wallet.sign()` rather than as a model error at the validation layer. The next field the network gains before the library does will fail exactly the same way. `validate()` should reject fields it does not know, or `Wallet.sign()` should say which layer is missing the field and which package version would have it.

Residual 2, P1 and NEW: the fix landed in `ripple-binary-codec` only. `xrpl` 5.2.0's own *model* layer still has no knowledge of close-ended vaults. Grep across `dist/npm` returns **zero** occurrences of `VaultKind` and zero of `SubscriptionDate`; `models/transactions/vaultCreate.d.ts` declares `Asset`, `Data`, `AssetsMaximum`, `MPTokenMetadata`, `WithdrawalPolicy`, `DomainID` and `Scale`, and nothing else; and `validateVaultCreate` checks none of the three new fields. The transaction succeeds only because unknown fields are passed through untyped — which is Residual 1 working in the developer's favour by accident.

The consequence is that on the current stable release the protocol's defining transaction has no type, no autocomplete, and no client-side validation of the `[180, 946708560)` investment-period window — a window `xrpl-py` 5.2.0b0 *does* validate, with a good message (U7). L7 is the read-side instance of the same gap, so `VaultCreate` and `VaultInfoResponse` are one gap rather than two. `XRPLF/xrpl.js#3456` "feat: Support Lending Protocol V1_1" is the PR that closes it and is still open (2026-08-31, last updated 2026-09-11). **Suggested fix:** merge #3456, or cut a 5.2.1 that adds the three fields to the `VaultCreate` interface and to `validateVaultCreate`.

Workaround for anyone still pinned to 5.1.0 (verified `tesSUCCESS` on Devnet): build `XrplDefinitions` from the live `server_definitions` response, then `rbc.encodeForSigning(tx, D)` + `ripple-keypairs.sign` + `rbc.encode(tx, D)` + `client.request({command:'submit', tx_blob})`. About twenty lines, and it bypasses `Wallet.sign` and `submitAndWait` entirely — which is precisely the API gap in L11.

L2 — no published version of `xrpl-py` can sign a `LoanSet` counterparty signature

P0 · `xrpl-py` 5.1.0 and 5.2.0b0 · NEW · fixed in `xrpl.js` 5.2.0, still open in Python

`fixCleanup3_4_0` introduced role-specific signing prefixes. The counterparty prefix is `CounterpartyTxSign = 0x43505400`. Both client libraries were signing with the plain transaction prefix `0x53545800`.

Verbatim error, live: `invalidTransaction: "fails local checks: Counterparty: Invalid signature."`

That message is the worst part of this bug. It reads as a key-management fault. Every developer who hits it will spend their first hour checking seeds, public keys and address derivation, because nothing in the error suggests the library is hashing the wrong prefix.

`xrpl.js`: fixed. In 5.1.0 and 5.2.0-beta.0, `dist/npm/Wallet/utils.js` was byte-identical and called `encodeForSigning(tx)` with no role. In 5.2.0 (and 5.2.0-beta.1) it routes properly — verified by reading the 5.2.0 tarball on 2026-09-12:

```
const SIGNING_ENCODERS = {
    transaction:  { single: (tx) => encodeForSigning(tx),             multi: … },
    counterparty: { single: (tx) => encodeForSigningCounterparty(tx), multi: … },
    sponsor:      { single: (tx) => encodeForSigningSponsor(tx),      multi: … },
};
```

and `ripple-binary-codec` 2.11.0 has `counterpartyTransactionSig: bytes(0x43505400)` (and `counterpartyTransactionMultiSig: 0x43504D00`) and exports both counterparty encoders. Confirmed working on the wire: `171FF3F1165F37753C3B946FA642F189BB9AEAE70BD3F845D58B47433C41B192`. Re-confirmed on rippled 3.4.0-rc5 during the event using stock `xrpl@5.2.0` `signLoanSetByCounterparty` with no patching: `58586F6D3BCAD18437D3CD1EE0F444CA346B7274BD7A38752F534F355D1B4D36` and `F993EB6BA7CCBE3B2461148DD77DFAE979A0A902AB937BB9D390050539BA7D37` (2026-09-12).

`xrpl-py`**: still broken, in every published version.** Verified 2026-09-12 against the published 5.1.0 and 5.2.0b0 distributions **as installed from PyPI**, not just the repository — PyPI `info.version` was still 5.1.0:

- `xrpl/transaction/counterparty_signer.py` → `compute_signature()` calls `encode_for_signing(tx_json)`.
- `xrpl/core/binarycodec/main.py` defines exactly four prefixes:

`_TRANSACTION_SIGNATURE_PREFIX = 0x53545800`, `_PAYMENT_CHANNEL_CLAIM_PREFIX = 0x434C4D00`, `_TRANSACTION_MULTISIG_PREFIX = 0x534D5400`, `_BATCH_PREFIX = 0x42434800`.

- `0x43505400` **does not appear anywhere in the package**, and there is no

`encode_for_signing_counterparty`.

So `sign_loan_set_by_counterparty()` cannot succeed, and there is no primitive to build a correct one from. **Python has no supported path to originate a loan on XRPL Devnet today.**

Workaround (verified `tesSUCCESS`**,** `93B8DC21800B24DD5886D76AFECB58E6B907398190490FD9C813A6388B7EC09C`**):**

```
tx_json = signed.to_xrpl()
msg = "43505400" + encode_for_signing(tx_json)[8:]     # swap the 4-byte prefix
tx_json["CounterpartySignature"] = {
    "SigningPubKey": broker.public_key,
    "TxnSignature": kp_sign(bytes.fromhex(msg), broker.private_key),
}
client.request(GenericRequest(method="submit", tx_blob=encode(tx_json)))
```

We asserted at runtime that the swapped message is byte-identical to `ripple-binary-codec`'s `encodeForSigningCounterparty()` output, then submitted both and got the same result.

Suggested fix: add `encode_for_signing_counterparty` / `encode_for_multisigning_counterparty` to `xrpl.core.binarycodec` and give `compute_signature` a `role` parameter, mirroring the `xrpl.js` fix. Also: change the rippled-side message. `"Counterparty: Invalid signature."` should say which prefix it expected, or at minimum that a role-specific prefix is required under `fixCleanup3_4_0`.

Related: `XRPLF/xrpl.js#3463` "Batch multi-sign: use `BCM` signing prefix (`fixCleanup3_4_0`)" is open, which suggests the role-prefix migration is incomplete in more than one place and might be worth auditing as a set rather than one role at a time.

L3 — `pip install xrpl-py` cannot construct a close-ended `VaultCreate`

P0 · `xrpl-py` **5.1.0 (PyPI** `latest` **as of 2026-09-12)** · **NEW**

```
VaultCreate(vault_kind=1, subscription_date=…, redemption_date=…, …)
# TypeError: VaultCreate.__init__() got an unexpected keyword argument 'vault_kind'
```

Fixed only in the `5.2.0b0` prerelease, which requires `--pre` or an exact pin, and which was published from the unmerged `XRPLF/xrpl-py#1034`. PyPI `info.version` was still `5.1.0` on 2026-09-12.

Combined with L2, Python is a full release behind `xrpl.js` on this amendment and cannot complete the lending happy path at all. The practical advice we ended up giving ourselves — and which the docs should probably give too, until L2 is fixed — is: do all signing in TypeScript, and restrict Python to read-only analytics via `GenericRequest`.

Suggested fix: merge #1034 and cut a stable release; or, if that is not imminent, say so on the Python tutorial tab, which currently presents Python as an equal path.

L4 — `LoanSet` is missing from `txToFlag`, the error names the wrong flag, and object-form `Flags` never reach the signer

P1 · `xrpl.js` **5.1.0, 5.2.0-beta.0, 5.2.0-beta.1 and** 5.2.0 stable **for the map;** all versions **for the signing path** · **NEW**

```
validate({ TransactionType:'LoanSet', Account:a, LoanBrokerID:b,
           PrincipalRequested:'1000000', Flags:{ tfLoanOverpayment:true } })
// ValidationError: Invalid flag tfLoanOverpayment. Valid flags are
//   {"1073741824":"tfInnerBatchTxn","tfInnerBatchTxn":1073741824}
```

`LoanSetFlags` and `LoanSetFlagsInterface` are exported, and the `LoanSet` interface declares `Flags?: number | LoanSetFlagsInterface`. The transaction is simply absent from the `txToFlag` map in `models/utils/flags.ts`. Verified in the **5.2.0 stable tarball** on 2026-09-12: the map contains `LoanManage`, `LoanPay` and `VaultCreate`, and not `LoanSet`. So this is an omission, not a design choice.

The error message is the second half of the finding. Telling a developer that `tfInnerBatchTxn` is the only valid flag for a `LoanSet` is actively misleading — it reads as a statement about the protocol rather than about a missing map entry.

Workaround: pass flags numerically. `Flags: 0x00010000` validates and works.

Suggested fix, and it is not one line. We originally wrote this up as a missing map entry. Re-testing against 5.2.0 stable on 2026-09-12 showed that adding `LoanSet` to `txToFlag` is necessary but **not sufficient, and on its own would make `LoanSet` worse**. `Wallet.sign()` never converts object-form `Flags` at all: it calls `validate(tx)`, which does not mutate the caller's object, and then `encode(tx)` directly — there is no `convertTxFlagsToNumber` anywhere on the signing path. So for the types that *are* in the map, the object form already fails, one layer later and with a worse message. On 5.2.0, `Wallet.sign` of a `LoanPay` with `Flags: {tfLoanLatePayment: true}` throws `Error: Cannot construct UInt32 from given value`, and so do `Payment` with `{tfPartialPayment: true}` and `VaultCreate` with `{tfVaultPrivate: true}`. Adding the map entry alone would downgrade `LoanSet`'s clear `ValidationError` to that opaque codec error.

The real fix is two parts: **(a)** add `LoanSet` to `txToFlag`; **(b)** have `Wallet.sign()` call `convertTxFlagsToNumber` on its working copy before encoding, so that the `Flags?: number | FlagsInterface` union the typings promise is actually reachable through the supported API. And when a type is absent from the map, say so (*"LoanSet has no registered flag map; pass Flags as a number"*) rather than printing the global fallback as if it were the answer.

L5 — dict-form `Flags` fail open to `0`, including the library's own documented spelling

P1 · `xrpl-py` 5.1.0 and 5.2.0b0 · ALREADY REPORTED: `XRPLF/xrpl-py#1016` (open since 2026-07-15)

```
LoanSet(..., flags={"TF_LOAN_OVER_PAYMENT": True}).to_xrpl()["Flags"]  # -> 0   (!!)
LoanSet(..., flags={"tf_loan_over_payment": True}).to_xrpl()["Flags"]  # -> 0
LoanSet(..., flags={"tfLoanOverpayment":  True}).to_xrpl()["Flags"]    # -> 0
LoanSet(..., flags=0x00010000).to_xrpl()["Flags"]                      # -> 65536  correct
```

No exception. No warning. We tested three spellings including `TF_LOAN_OVER_PAYMENT`, which is the library's own `LoanSetFlag` member name, and all three produced `Flags: 0`.

Issue #1016 already describes the root cause exactly — `interface_to_flag_list()` appends `0` instead of raising — and calls it "a fail-open condition in transaction construction". We are not opening a duplicate.

What we are adding, as a comment on #1016: the consequence in a lending context. A developer who asks for an overpayment-enabled loan gets a loan created **without** the overpayment permission, no error is raised anywhere in the stack, and the mistake is only discoverable by reading the created `Loan` object's `Flags` back off the ledger. In a money primitive that is a correctness hazard, not an ergonomics one, and we would argue it justifies a higher priority than a general flags bug would get.

L6 — the typed `ledger_entry` request model cannot select a `Loan` or a `LoanBroker`

P1 · both libraries · NEW

The three lending ledger entries are reachable from the server and unreachable from the typed request models.

Python: `xrpl/models/requests/ledger_entry.py` defines only `vault` among the new types; `LedgerEntryType` has `SINGLE_ASSET_VAULT = "vault"` but no `LOAN` or `LOAN_BROKER`.

```
LedgerEntry(loan="65D1149BDD…")
# TypeError: LedgerEntry.__init__() got an unexpected keyword argument 'loan'
```

xrpl.org documents both (`loan` taking `loan_broker_id` + `loan_seq`; `loan_broker` taking `owner` + `seq`), and the server accepts both at runtime — we read live `Loan` objects this way.

TypeScript: `models/methods/ledgerEntry.d.ts` in **5.2.0 stable** types `mpt_issuance`, `mptoken`, `amm`, `credential`, `deposit_preauth`, `did`, `directory`, `escrow`, `offer`, `payment_channel`, `ripple_state`, `ticket`, `nft_page`, the bridge selectors, `delegate` and `sponsorship` — and **none of** `vault`, `loan` **or** `loan_broker`. Because `BaseRequest` declares `[x: string]: unknown` (`models/methods/baseMethod.d.ts` line 4) the omission is silent:

```
// tsc 5.6.3 --strict --skipLibCheck against xrpl@5.2.0
const a: LedgerEntryRequest = { command:'ledger_entry', loan:'ABC'  }   // no error, no autocomplete
const b: LedgerEntryRequest = { command:'ledger_entry', lonn:'typo' }   // ALSO no error
const c: number = "definitely a string"                                 // error TS2322 — checking is active
```

Workarounds: `GenericRequest(method="ledger_entry", loan=…)` in Python (verified, returns `LedgerEntryType: Loan`); in TypeScript, just pass the field and accept that you have no type safety.

Suggested fix: add the three selectors to both models. In TypeScript this also argues for removing `BaseRequest`'s index signature, or narrowing it, because it currently defeats the purpose of having typed request models at all. A related open PR exists: `XRPLF/xrpl.js#3230`.

L7 — `VaultInfoResponse` omits the four fields that define a close-ended vault

P1 · `xrpl.js` **5.2.0 stable** · **NEW**

`models/methods/vaultInfo.d.ts` in the 5.2.0 tarball types `result.vault` with `Account`, `Asset`, `AssetsAvailable`, `AssetsTotal`, `LossUnrealized`, `ShareMPTID`, `WithdrawalPolicy`, `Scale`, the nested `shares` issuance, and so on — but **no** `VaultKind`**, no** `LEVersion`**, no** `SubscriptionDate`**, no** `RedemptionDate`.

The live server returns all four. `vault_info` on vault `6717B511…ACDD773C` at 2026-09-12 07:37 UTC returned these keys:

```
Account, Asset, AssetsAvailable, AssetsTotal, Data, Flags, LEVersion, LedgerEntryType, LossUnrealized,
Owner, OwnerNode, PreviousTxnID, PreviousTxnLgrSeq, RedemptionDate, Sequence, ShareMPTID,
SubscriptionDate, VaultKind, WithdrawalPolicy, index, shares

with VaultKind: 1, LEVersion: 1, SubscriptionDate: 842466494, RedemptionDate: 842552594
```

So the typed response for the protocol's own vault read API cannot express the vault kind, the accounting version, or either phase boundary. A TypeScript consumer has to cast away the type to determine whether a vault is even capable of hosting a loan.

This is the read-side half of L1 residual 2. The same four fields are missing from the write side (`models/transactions/vaultCreate.d.ts`), so `VaultCreate` and `VaultInfoResponse` are one gap, not two, and both close when `XRPLF/xrpl.js#3456` merges.

Precedent: `XRPLF/xrpl.js#3197` "`VaultInfoResponse` missing the `Data` field" — closed. Same class of defect; this is the `LendingProtocolV1_1` instance of it.

L8 — `get_amount_value()` returns `float`

P2 · `xrpl-py` · **ALREADY REPORTED:** `XRPLF/xrpl-py#978` **(open since 2026-04-17)**

```
get_amount_value("10000000000000001")   # -> 1e+16
get_amount_value("9999999999999999")    # -> 1e+16
```

The signature is `-> float`. **What we are adding to #978:** this is in a lending library whose `Number` fields carry up to 19 significant digits, and we have live captures to prove the digits are real — `PeriodicPayment` came back as `"3333346.017258355048"` and `"2500035.673617762465"` on real loans. Applying `get_amount_value()` to `AssetsTotal`, `LossUnrealized`, `PrincipalOutstanding`, `DebtTotal` or `PeriodicPayment` loses money silently. It should return `Decimal`.

For completeness on the same theme: `Number` round-trips through `ripple-binary-codec` normalise to 19 significant digits and can come back in scientific notation — `"9999999999999999999"` → `"1e19"`, `"1.2345678901234567890123456789"` → `"1.234567890123456789"`. `BigInt("1e17")` throws; `new Decimal("1e17")` does not. This is worth a sentence on the `Number` type page, because the natural reflex for an integer-looking string is `BigInt`.

L9 — `DEFAULT` fields typed as required, and a defaulted loan deletes four of them

P2 · `xrpl.js` · **ALREADY REPORTED:** `XRPLF/xrpl.js#3154` **(open since 2025-12-10)**

What we are adding: the concrete lending case. On `tfLoanDefault`, rippled deletes `PrincipalOutstanding`, `TotalValueOutstanding`, `PaymentRemaining` and `NextPaymentDueDate` from the `Loan` object, leaving `Flags 196608` plus `StartDate`, `PeriodicPayment` and `PreviousPaymentDueDate`. All four deleted fields are typed as required in the ledger model, so a consumer that trusts the types gets `undefined` where the compiler promised a value. It crashed our indexer.

Also in this family: `models/ledger/Vault.d.ts` declares `LedgerIndex: string` as required, and rippled returns `index`. `Vault` is the only ledger model that does this.

L10 — `Loan.OverpaymentFee` typed as a string; the wire type is `UInt32`

P2 · `xrpl.js` **5.2.0** · **NEW**

`models/ledger/Loan.d.ts` line 15: `OverpaymentFee?: XRPLNumber` (a string). `definitions.json` declares `OverpaymentFee` as `UInt32` (nth 64), and `models/transactions/loanSet.ts` types the same field as `number` and validates it with `isNumber`. Passing the string the ledger model says you will receive gives `ValidationError: LoanSet: invalid field OverpaymentFee`. One of the two declarations is wrong.

L11 — no custom-definitions API on the high-level client, in either language

P2 · `xrpl.js`, `xrpl-py` · **SPECIFIED BUT UNMERGED: four open pull requests, none merged** — `xrpl.js#3229` **(2026-03-23) and** `#2683` **(2024-04-23), both titled "feat(xrpl): custom definitions support",** `#2597` **"Support for native asset & hash in custom Definitions" (2023-12-05), and** `xrpl-py#899` **"feat(binarycodec): add DefinitionsRegistry with contextvars support" (2026-01-30)**

`ripple-binary-codec` takes an optional `definitions` argument on `encode`, `decode` and `encodeForSigning`, and exports `XrplDefinitions`. `xrpl.js`'s `ClientOptions`, `Wallet.sign()`, `xrpl.encode()` and `xrpl.encodeForSigning()` take none — `xrpl.encode.length === 1`. `xrpl-py` has no public API for it either; you have to monkey-patch `_DEFINITIONS`, `_FIELD_INFO_MAP` and `_FIELD_HEADER_NAME_MAP`.

What we are adding: a dated cost. The escape hatch you need when the published library lags the network is exactly the one that is missing, and L1 is the example — for 17 days the only way to build a vault from the published stable release was to fetch `server_definitions`, construct `XrplDefinitions` by hand, and abandon `Wallet.sign` and `submitAndWait` for the whole application. Three separate attempts to add this API to `xrpl.js` have been open for 6, 29 and 33 months respectively, and a fourth has been open against `xrpl-py` for 7. Nobody is refusing to build it; it is written, four times over, and unmerged. On a ledger that ships amendments faster than it ships client releases, this is the difference between a one-line pin and a rewrite of your signing layer.

L12 — the library writes to the console on the success path, and the vendor's own tutorial patches it away

P2 · `xrpl.js` **5.2.0** · **NEW**

`sugar/autofill.js` line 225 emits an unconditional `console.warn` on every `LoanSet`:

```
For LoanSet transaction the auto calculated Fee accounts for total number of signers
the counterparty has to avoid transaction failure.
```

The evidence that this is real friction is not our opinion. It is in Ripple's own code sample — `_code-samples/lending-protocol/js/lendingSetup.js`, lines 266–267:

```
// Suppress unnecessary console warning from autofilling LoanSet.
console.warn = () => {}
```

The tutorial globally disables `console.warn` for the rest of the process to keep its output readable. When the vendor's own example has to silence your logging, the logging is wrong. A library should not write to the console on a success path; this belongs behind a logger or should be removed.

A smaller note on the same line: the message says the calculated fee *accounts for* the counterparty's signers. In our runs `autofill()` returned `Fee: '2'` for a two-signature `LoanSet`, and we defensively raised it every time rather than find out. Either the fee is being raised and the message is unnecessary, or it is not and the message is wrong; from outside the library we cannot tell which, and that ambiguity on a money transaction is itself the problem.

L13 — `VaultCreate` autofill charges the owner reserve and bypasses `maxFeeXRP`

P2 · `xrpl.js` · **SPECIFIED BUT UNMERGED:** `XRPLF/xrpl.js#3430` **"fix: use base fee for** `VaultCreate` **autofill", an open pull request since 2026-08-10, last updated 2026-09-01**

`sugar/autofill.js` groups `VaultCreate` with `AccountDelete` and `AMMCreate` in `isSpecialTxCost`, takes `baseFee` from `fetchOwnerReserveFee`, and then computes `totalFee = isSpecialTxCost ? baseFee : BigNumber.min(baseFee, maxFeeDrops)` — so `maxFeeXRP` is bypassed. Observed live on Devnet 3.4.0-rc5, 2026-09-12: `Fee: "200000"` on a `VaultCreate` (`2ED37F15D02E0899614EB8FCEB7F19FE26B5F3C3D9E15EDF931F0FF98D1AA536`) while the `LoanBrokerSet`, `VaultDeposit`, `LoanBrokerCoverDeposit` and `LoanManage` submitted from the same account within the same minute all autofilled to `Fee: "1"`, and the `LoanSet` to `Fee: "2"` (`1771BD0AF4D3EE1A095A79170005DE2E2D681F818D08BE2327D8EF4932DE8E1B`). Devnet `base_fee` and `open_ledger_fee` were both 1 drop. Mentioned here only because it surprises everyone and the fix is written — it has been open for a month.

---

7. UX FRICTION

U1 — `tecNO_PERMISSION` means four different things on the lending path

P1 · rippled · NEW

TRANSACTION	WHAT TECNO_PERMISSION MEANT	EVIDENCE
`LoanBrokerSet`	the vault is not close-ended	`86AC8182`… (owner + open-ended)
`LoanBrokerSet`	you are not the vault owner	`A260515D`…, `FCF6DC50`…
`LoanSet`	the loan matures after the vault's `RedemptionDate`	interval 28251 × 3 → `tesSUCCESS`; 28751 × 3 → `tecNO_PERMISSION` (`055E2AE47EF09A12D776C4729FC543A35011490813A41DD02D4682165E21F068`)
`LoanManage`	the loan is already impaired (`tfLoanImpair` is not idempotent)	`F392154FAF4E20502A914B9EB6C2A941A12F85327DF7C841C23FB6E7E5D91571`

Three of the four cells in the `LoanBrokerSet` matrix return the identical, undifferentiated code, so the result does not tell you which of two independent preconditions you violated.

This cost us real time in a way worth describing, because it is the failure mode this code creates. One of our probes submitted a `LoanSet` with the broker as `Account`, got `tecNO_PERMISSION`, and concluded the role order was reversed. It was not — the create-a-loan tutorial says explicitly that "these fields can be swapped", and two other probes of ours succeeded with the broker as `Account`. The actual cause was the loan's maturity overrunning `RedemptionDate`. We nearly filed a false report on the strength of an overloaded error code, and caught it only because we cross-checked three probes against each other. We have listed it in §10 as withdrawn.

Suggested fix: distinct result codes for distinct preconditions where one is available, and otherwise surface the transactor's existing `JLOG` reason in the transaction metadata. rippled already computes a human-readable reason — `"LoanBroker requires a closed-ended Vault."` — and then throws it away. Returning it would collapse a class of multi-hour debugging sessions into a single read.

And this is already half-built. `XRPLF/rippled#7848` "feat: Experimental: Add more specific error messages in submit/simulate" (open since 2026-07-22) threads the transactor's `JLOG` line into a new `engine_result_reason` field on `submit` and `simulate`; its own sample output shows `engine_result_reason: Too many or too few signers in signer list.` marked as the new addition. Applied to `LoanBrokerSet`, that PR alone turns our two-hour debugging session into one read — it would have returned "LoanBroker requires a closed-ended Vault." directly. Our only ask on top of it is that `tec` results carry the reason into **metadata** as well, so it survives for anyone reading the transaction back later rather than only for the submitter.

U2 — `tecINSUFFICIENT_FUNDS` cannot distinguish "you do not own that" from "the vault has no cash", and there is no partial fill

P1 · rippled `VaultWithdraw` **· NEW**

`VaultWithdraw.cpp` has two independent guards that return the same code: `accountHolds(...) < sharesRedeemed` (the holder's share balance) and `*assetsAvailable < assetsWithdrawn` (vault liquidity). Proven with two live transactions on the same vault:

REQUEST	WHICH GUARD FIRED	RESULT	HASH
20 XRP, holder has 20,000,000 shares, vault has 5 XRP available	liquidity	`tecINSUFFICIENT_FUNDS`	`52F49C182F91E363BE59037FE3E3E2AB18E03B9E72C9488BCCA87DD2BF9DC1D7`
21 XRP, more than the holder's shares are worth	share balance	`tecINSUFFICIENT_FUNDS`	`F24B4469557FDCD5B45E1574364CAA5FED2623DD00F5EB429677CCF87B6CB63B`

A wallet cannot tell a user which of those two it was without separately reading the Vault SLE and the holder's `MPToken`. We had to write UI copy for this, and the only safe wording is "rejected by the ledger", which is not helpful to anyone.

And there is no partial fill. A depositor asking for 20 XRP when 5 XRP is available gets `tecINSUFFICIENT_FUNDS` and zero drops; the vault state afterwards is byte-identical. Requesting *exactly* `AssetsAvailable` succeeds — the guard is a strict `<` — verified at `AF294F4A3DBC3D0166E7B0B739E2670E23B0E24501A0799A8F0BEAFEDC379E96`, which paid exactly 5,000,000 drops and drove availability to zero. So every client must compute the maximum withdrawable amount itself, from two objects, before every exit attempt, or the user watches their withdrawal bounce.

Two adjacent codes in the same area are worth documenting **on the pages**, though we now understand the design. A fully drained vault returns `tecPRECISION_LOSS` for an asset-denominated request (`1E972523FCCDD2F254839DD293DD0F57CD5957F9902E4BE48C12B7D81FF745B8`), while a **share**-denominated request against the same vault returns `tesSUCCESS` and pays **zero drops** (`9C64649C496C8F645AD059240F72ED7AA7E384C10CA1E32A1DCDE7CC1FC6E45C` — 20,000,000 shares burned, `xrpDelta` = −100, the fee). Both are intended: `XRPLF/rippled#7950` "fix: Reject `VaultWithdraw` fixed-share amounts that round to zero" (merged 2026-08-19, gated on `fixCleanup3_4_0`) added the `tecPRECISION_LOSS` guard to the fixed-shares branch and

deliberately kept zero-value withdrawal legal when the pool's effective value is genuinely zero, because a fully impaired vault must still let holders exit. That is a good decision and we would not change it. It is simply not on `vaultwithdraw.md`, and a wallet that surfaces "success" on a withdrawal that paid nothing will be accused of losing funds.

U3 — seven rate fields are in 1e-5 units, and the only written-down source is a constant inside `node_modules`

P1 · XLS-66 · PARTIALLY TRACKED: `XRPLF/XRPL-Standards#625` **(open since 2026-09-09)**

The unit is neither basis points nor ppm. It is 1e-5 — one hundred-thousandth, 0.001%. `100000` is **100.000%**.

FIELD	CAP	10% IS WRITTEN AS
`InterestRate`, `LateInterestRate`, `CloseInterestRate`, `OverpaymentInterestRate`, `OverpaymentFee`, `CoverRateMinimum`, `CoverRateLiquidation`	100000	`10000`
`ManagementFeeRate` (UInt16)	**10000**	`1000`

A developer who writes `100000` meaning 10% creates a loan at 100% APR. We confirmed the unit numerically on the wire rather than trusting any document: a 10,000,000-drop loan at `InterestRate: 100000` with `PaymentInterval: 28251` produced `PeriodicPayment 3339307.338128906896`, and the standard annuity formula with `r = PaymentInterval / 31_536_000 × (InterestRate / 100000)` reproduces that to ten significant figures (`8DE13240603C8787A739BC3EE47F0392937C37B942EAEB3A91D7DF47038F63D5`). The implied year is 31,536,000 seconds, which is also undocumented.

The most useful written source we found for any of this was `node_modules/xrpl/dist/npm/models/transactions/loanSet.js` — the client-side validation constants (`MAX_INTEREST_RATE = 100000`, `MAX_MANAGEMENT_FEE_RATE = 10000`, `MIN_PAYMENT_INTERVAL = 60`, `GracePeriod <= PaymentInterval`). Reading the library's compiled validators was more productive than reading the specification, which is a signal worth acting on.

PR #625 clarifies `CoverRateMinimum` specifically. **Our contribution, as a comment there:** the unit applies to seven fields, `ManagementFeeRate` has a different cap in a different integer type, and the year constant is 31,536,000 s. A one-sentence note in each field table, or a rename to something like `*RateTenthBips`, would close the whole class.

U4 — fields that may not be set to their default value, in a client that requires them

P1 · rippled + `xrpl.js` · NEW (the layer contradiction); the `Scale` range sub-case is already reported as `xrpl.js#3435`

rippled rejects an explicitly-default `Scale` at local-check time, before consensus:

```
error: "invalidTransaction"
message: "Field 'Scale' may not be explicitly set to default."
```

This bites on `VaultCreate` (do not send `Scale: 0`; omit it) and on `OracleSet`'s `PriceDataSeries`. But `xrpl.js`'s `validate()` for `OracleSet` throws `PriceDataSeries must have both AssetPrice and Scale if any are present` when you omit it. The two layers have **opposite** rules, so `Scale: 0` is unreachable through `Wallet.sign` / `submitAndWait` and you must raw-sign to express it. That contradiction is the finding, and it searched clean across rippled, `xrpl.js` and XRPL-Standards.

(Separately, and already filed: `xrpl.js` caps `Scale` at 0–10 — `SCALE_MAX = 10` in `oracleSet.js` — while rippled accepts up to 20. That is `XRPLF/xrpl.js#3435` "OracleSet validation rejects valid Scale values 11-20 (SCALE_MAX should be 20, not 10)", open since 2026-08-12. We reproduced both ends on Devnet — 20 → `tesSUCCESS` `E95ABBCE6E6E3F720A14253BDBE17BF535A5210CAE8CC6A3E852B4E5D3EDCB80`, 21 → `temMALFORMED` — and added those hashes to the issue rather than opening a second one. The `Scale: 0` contradiction above is a different boundary and is filed separately.)

Suggested fix: pick one convention. Either accept an explicit default and ignore it, or document the "omit to default" rule on every field it applies to and make the client validators agree with it.

U5 — absent means zero, everywhere, with no normalising option

P1 · rippled read APIs · ALREADY REPORTED (`XRPLF/xrpl-dev-portal#3612`), with a new ask

Many fields are not stored on a ledger object while they hold their type default, so they are absent from objects and from metadata alike. A healthy vault has **no** `LossUnrealized` **key at all** — not `"0"`. So does a vault with no `Scale`, a broker with no `CoverAvailable`, a loan whose `ManagementFeeOutstanding` rounds to sub-drop.

Every read in a lending application therefore needs `v.AssetsAvailable ?? '0'` on every numeric field, and the failure mode when you forget is `undefined` arithmetic producing `NaN`, not an exception. Combined with X1, where the same absence leaves `PreviousFields` empty, this is the single most common source of crashes we hit, and it is mentioned on no page today.

It has, however, been an open ticket since 2026-04-15, filed by Ripple's own documentation lead. `XRPLF/xrpl-dev-portal#3612` "Several fields (esp. lending-related entries) are actually optional", by `mDuo13`, already proposes our cheap fix in almost these words: *"The* `LoanBroker` *ledger entry's* `DebtTotal` *is one of several fields that's listed as required in XLS-66 but implemented as* `soeDEFAULT` *in the source which means it's effectively optional, but treated as 0 if omitted… Any* `soeDEFAULT` *field is Optional but the description should note that, if omitted, it is treated as 0."* It is explicitly scoped to the lending entries, and it is the docs-side twin of L9. We are not duplicating it.

What we are adding, as a comment there. First, the rule applies identically to transaction **metadata**, which is finding X1 and is the expensive half — a reference-page note would not have saved our indexer. Second, the failure mode is `undefined` arithmetic producing `NaN` rather than an exception, so the bug surfaces as a wrong number downstream rather than as a crash at the read. Third, the ask #3612 does not make: **an opt-in request parameter** (`"defaults": true`, say) that materialises default-valued fields in the response. Consumers who care about bandwidth keep today's behaviour; consumers who care about correctness get an object whose shape does not depend on its contents.

U6 — a `tem`/`tel` rejection throws an error carrying no structured result, and three failure layers surface three different ways

P2 · `xrpl.js` · **NEW**

An earlier draft of this finding said `submitAndWait()` throws on `tec`. It does not, and D2 in this same document depends on the opposite being true. D2 is right, and we have corrected this section rather than leave two of our own findings contradicting each other.

`client.submitAndWait()` returns `tec` results correctly — a `tec` is applied, validated, and comes back with full metadata. Verified on 2026-09-12 against rc5: a `tecNO_DST_INSUF_XRP` returned normally with `meta.TransactionResult` set and `validated: true` (`B857FE4BD45D6D4A1A86092607756E712BFE180A1C727CA152484FF4A80F6A9E`). In the source, `waitForFinalTransactionOutcome` in `sugar/submit.js` returns the response as soon as `result.validated` is true and never inspects `TransactionResult`.

The gap is one layer up. A `tem` or `tel` is never applied, and `submitAndWait` surfaces it as a thrown `XrplError` whose message embeds the code as free text — `"Transaction failed, temREDUNDANT: The transaction is redundant."` — with `e.data` **undefined**. There is no structured field to read the code from, so every wrapper ends up string-matching the message. (Our original capture was `"Transaction failed, temINVALID: The transaction is ill-formed"`, which is the same shape.)

Underneath, there are three distinct failure layers and they surface in three different ways:

1. a codec error throws synchronously from `encode()` with no code;
2. a rippled local check comes back as a **request** error (`error: "invalidTransaction"`) with a human

message and **no** `engine_result`;

1. a real engine result arrives as `engine_result`.

Any error-handling wrapper that does not catch all three will report a local check as a network failure. Every probe in this project ended up abandoning `submitAndWait` for `client.request({command:'submit', tx_blob})` plus polling `{command:'tx'}` — about twenty lines — because that path returns the code cleanly for all three and distinguishes `tem`/`tel` (never applied, no hash to poll, returns in ~0.2 s) from `tec` (applied, has metadata, costs a ~5 s ledger round trip and a real fee).

Suggested fix: attach the engine result and the preliminary submission response to the thrown error as structured fields, rather than only interpolating them into the message string. And the distinction between the three layers deserves a page of its own; it is the first thing anyone writing a submission wrapper needs, and we could not find it written down.

U7 — client-side validators disagree with the server and cannot be bypassed through the supported API

P2 · `xrpl.js` · **NEW**

`validate()` runs inside `Wallet.sign()`, so there is no supported way to ask the server what it actually does. Where a validator is stricter than the server you get a helpful local error; where it is *wrong* — the `Scale` cases in U4, or `LoanSet`'s missing flag map in L4 — you are blocked from a legal transaction with no override.

(The good example of the first kind is `xrpl-py`'s, not `xrpl.js`'s, and we had it attributed to the wrong library until re-verification. `xrpl-py` 5.2.0b0 raises `XRPLModelException {"redemption_date": "redemption_date - subscription_date must be within [180, 946708560) seconds."}` and defines `MIN_INVESTMENT_PERIOD` / `MAX_INVESTMENT_PERIOD` with docstrings citing rippled's `kMinInvestmentPeriod`. That is exactly the right behaviour and we want to name it. `xrpl.js` 5.2.0 has no equivalent check — `validate()` accepts a `RedemptionDate` 100 seconds after `SubscriptionDate` without complaint, because its `VaultCreate` model does not know either field exists. Which is its own finding; see L1 residual 2.)

Suggested fix: a documented `skipValidation` option on `Wallet.sign()` and `submitAndWait()`, or export the raw-signing path as a supported API rather than leaving every team to rediscover `encodeForSigning` + `ripple-keypairs.sign` + `encode` independently. We wrote that helper four times across six probes.

U8 — the protocol's minimum feedback loop is about three minutes, and there is no way to shorten it

P2 · rippled · NEW

Closed-ended vaults are mandatory for lending, so every lending integration test now inherits a wall-clock phase machine:

CONSTRAINT	MEASURED
`SubscriptionDate` must be **in the future** at `VaultCreate`	past → `tecEXPIRED` (`54C94DAADABDFEC8BEC88DD32A8E0A6135E4F5396B2BBFA34733E22769509087`)
Minimum practical lead	+6 s → `tecEXPIRED`; +12 s → `tesSUCCESS`. Two data points, not a bisection; we used 45–60 s
`RedemptionDate - SubscriptionDate`	exactly **[180, 946708560)** seconds; 179 → `temMALFORMED`, 180 → `tesSUCCESS` (`0C6127F4E9042C5EB1DD85A7274FC0748F19D9D1839256459ED5D206D9A14D06`)
`LoanSet` before `SubscriptionDate`	`tecTOO_SOON`
Impairment requires a late payment	`tecTOO_SOON` until `close_time >= NextPaymentDueDate`; minimum `PaymentInterval` is 60 s
Default requires grace	`tecTOO_SOON` until `close_time > NextPaymentDueDate + GracePeriod`

End-to-end measurements from cold faucet accounts: zero-to-funded-loan **88.5 s** and **143.4 s** on two independent runs; full vault lifecycle to first successful withdrawal **200.7 s**; full originate→pay→impair→default cycle **395.5 s**; a pre-baked distress scenario **453 s** to build and 64 s to run. Of the 453 s, only about 75 s is transaction work — the remaining ~378 s is the phase clock.

Individual transactions are fast and predictable (3.5–7.8 s to validation, dominated by ledger close), so this is not a performance problem. It is that the protocol's state machine is keyed to wall-clock time with a 180-second floor, and a test network offers no way to advance it.

Suggested fix, and we think this is the most valuable DX item in this section: a Devnet-only admin method to advance a vault's phase, or a reduced `MIN_INVESTMENT_PERIOD` under a devnet-only rule. Without it, every CI suite for this protocol is minutes long per case, which in practice means teams will not write one. `xrpl.js`'s own `lendingProtocol.test.ts` is a case in point — it builds an open-ended vault and therefore does not reproduce against public Devnet at all.

U9 — phase gates evaluate against ledger `close_time`, which lags wall clock, and nothing says so

P2 · docs · NEW

Every phase predicate in rippled uses the applying ledger's `close_time`, and preflight sees the **parent** ledger's close time. Devnet's `close_time_resolution` is 10 seconds and `close_time` lags real time by up to ~10 s, advancing in irregular 1–9 s jumps.

Consequence, measured: a `LoanSet` submitted at exactly `SubscriptionDate + 0 s` still returned `tecTOO_SOON`; at `+9 s` the code changed. A scripted demo that sleeps on `Date.now()` will fail intermittently, and the failure looks like a race condition in your own code.

Suggested fix: one line on each phase-gated transaction page: *"This condition is evaluated against the close time of the ledger in which the transaction applies, not the submitter's clock. Poll* `{command:'ledger', ledger_index:'validated'}` *and read* `ledger.close_time`*."*

U10 — one transaction, three field-shape conventions, and two write-once fields behind an undifferentiated error

P2 · XLS-66 / `xrpl.js` **· NEW; the** `VaultID` **sub-case is already specified in** `XRPL-Standards#497` **and implemented in** `rippled#6528`**, both unmerged**

On a single `LoanBrokerSet`:

- `CoverRateMinimum` and `CoverRateLiquidation` are `UInt32` **numbers**. Passing a string throws

`ValidationError: invalid field CoverRateMinimum`.

- `DebtMaximum` is a `Number` field passed as a JSON **string**.
- `ManagementFeeRate` is a `UInt16` number with a different cap from the other rates (U3).
- The two cover rates must **both** be zero or **both** non-zero, or you get `temINVALID` with no indication

which field is at fault.

- Both cover rates are **write-once**. A `LoanBrokerSet` that changes either on an existing broker returns

`temINVALID` — again with no hint. Changing only `DebtMaximum` on the same broker succeeds (`EC043B5A66B3FEE9F638C90D2C78F36CBB46C990DF47906BB146EF3B4B578CE6`).

- `VaultID` is required on **every** `LoanBrokerSet`, including an update (`Field VaultID is required but

missing), which the typings do not reflect. This one is already solved on paper and waiting: XRPLF/XRPL-Standards#497 "Make VaultID conditional on LoanBrokerSet" and its implementation XRPLF/rippled#6528 have both been open since 2026-03-11 — updated 2026-09-11 and 2026-09-09 respectively — and would make VaultID required on create and temINVALID` on update, on the stated grounds that the vault association is fixed at creation and cannot be changed. We mention it only to note that the six-month-old fix would also remove one of the three field shapes above.

Also, on Devnet `network_id` is 2, which is below 1024, so the `NetworkID` field **must be absent** — including it gives `telNETWORK_ID_MAKES_TX_NON_CANONICAL`. Anyone hand-rolling autofill from a mainnet example hits this first.

None of these are hard once you know them. All of them cost a ledger round trip to discover.

---

8. MISSING PRIMITIVES

M1 — no state proof on any read API, so nothing off-ledger can verify a fact about XRPL ledger state

P0 · rippled · NEW · and we want to frame this one carefully

Related, already open: `XRPLF/XRPL-Standards#611` reports that XLS-0065/0066 and XLS-0096 are silently non-composable at the accounting level, from live Devnet execution. Our finding sits upstream of that one — it is about proofs, not balances — but they are the same wall seen from two sides.

We spent real time trying to build privacy-preserving credit on XRPL and concluded it is currently unbuildable. Not hard — unbuildable. The striking part is that **every component already exists somewhere in the ecosystem**; they simply cannot be composed. We are reporting this as a primitive gap rather than a complaint because we think the gap is one API away from closing.

The three facts, each verified live on 2026-09-11.

1. **rippled exposes no state or Merkle proofs.** `ledger_entry` has no `proof` parameter; passing

`proof: true` alongside `binary: true` is silently ignored and the response contains only `{index, ledger_hash, ledger_index, node_binary, status, validated}` — no branch, no path. `LedgerEntry.cpp` contains zero occurrences of "proof", and no proof-related command exists in rippled's handler registry. So there is no way for a light client, a bridge, or a zero-knowledge circuit to bind a statement to XRPL ledger state. Anything that claims to is really trusting whoever served the JSON.

1. **The generic verifier and the lending amendments live on disjoint networks.** A generic

Groth16 / RISC Zero verifier does run natively on an XRPL ledger — on `groth5.devnet.rippletest.net`, rippled `3.0.0-b1`, `network_id 1256`, with `SmartEscrow` enabled and BN254 host functions. We queried it live. That network has **no** `SingleAssetVault`, **no** `LendingProtocol`, **no** `ConfidentialTransfer`. Standard Devnet has all three and **no** `SmartEscrow`. You cannot put a verifier and a vault on the same chain today.

1. **The confidential primitive is reachable, but it cannot be pointed at ledger state.** XLS-0096

Confidential MPT is `Final`, enabled on standard Devnet — its five transaction types are in `server_definitions` (`ConfidentialMPTConvert` 85, `ConfidentialMPTMergeInbox` 86, `ConfidentialMPTConvertBack` 87, `ConfidentialMPTSend` 88, `ConfidentialMPTClawback` 89) — and it now has client bindings in all three major SDKs. We had written here that it has no SDK in any language; that was wrong, and it was wrong about the very tarball we quote line numbers from elsewhere in this document. `xrpl@5.2.0` ships the five transaction models and exports `deriveConfidentialKeypair`, `getConfidentialBalance`, `prepareConfidentialConvert`, `prepareConfidentialConvertBack`, `prepareConfidentialMergeInbox`, `prepareConfidentialClawback`, `prepareConfidentialSend` and `prepareConfidentialBatch` — the proving helpers included (`XRPLF/xrpl.js#3364`, merged 2026-08-20; we confirmed the exports against the installed package). `xrpl-py` has carried the five `confidential_mpt_*` models since `XRPLF/xrpl-py#919`, and `XRPLF/mpt-crypto` implements the proof system. So the confidential rail is buildable today. What is not buildable is any statement binding a confidential computation to a *fact about the ledger* — because of (1).

Why this matters for lending specifically. Ripple, Clearpool and Cicada announced an RLUSD-denominated institutional lending platform on XRPL on 2026-08-20/21, built on exactly these two amendments, with a curator-driven risk structure and development "testing on XRPL Devnet". Institutional credit runs on selective disclosure: a borrower proves a covenant is met without publishing the balance sheet; an LP proves accreditation without publishing an identity. XRPL has the confidential-value primitive (XLS-0096), the attestation primitive (XLS-70 Credentials, which we used, and which is live on Mainnet), and a verifier that runs on a ledger. It has no way to prove anything *about* its own state to any of them.

Asks, ranked by cost to Ripple:

1. **A** `proof` **option on** `ledger_entry` **returning the SHAMap inclusion branch.** This is a read-only,

non-consensus, amendment-free addition to a handler that already walks the map, and it is the single highest-leverage API addition available in the protocol. It unlocks light clients, trust-minimised bridges, and any circuit that wants to say something about ledger state — including "this vault's `LossUnrealized` was X at ledger N", which is the thing our own product would most

like to be able to prove.

1. **One devnet carrying both amendment sets**, even temporarily. The combination is currently untestable by

anyone, which is also what `XRPL-Standards#611` runs into from the accounting side.

M2 — no native read of what a vault share is worth

P1 · rippled read APIs · NEW

The correct per-share value is `(AssetsTotal - LossUnrealized) / shares.OutstandingAmount`. Three things make that harder than it should be:

- There is **no** `SharesTotal` on the `Vault` object. `OutstandingAmount` lives on the share

`MPTokenIssuance`. Credit where it is due: `vault_info` returns both the vault and the issuance in one call, which is exactly right and saved us a round trip.

- Nothing computes the ratio, and the naive `AssetsTotal / shares` reads **1.000000** straight through a

19.61% impairment (X1). Every consumer will write this formula, and the ones who write it wrong will not find out.

- **One valuation rule cannot be reproduced off-ledger at all.** `VaultWithdraw` waives the unrealised loss

when the withdrawer is the vault's **sole shareholder**. We proved this with a controlled A/B that we think is worth reading: a sole holder submitted a 1-share withdrawal against a vault with `AssetsTotal` 15,000,001 / `AssetsAvailable` 1 / `LossUnrealized` 15,000,000 and got `tecINSUFFICIENT_FUNDS` (`B23954920C0057036EE84EB566DE64B187CFD8A019335820BCF60E7100C3A4C5`). She then sent **exactly one share** to another account by `Payment` — changing no vault field, no `AssetsTotal`, no `OutstandingAmount` — and resubmitted the **byte-identical** transaction. The result code flipped to `tecPRECISION_LOSS` (`4FD9B7AF2E6E94C71CD6193BE51C992962A735C1C2E514BA733B47284EED535C`), because the un-waived basis is 15,000,001 − 15,000,000 = 1 drop and one share now rounds to zero assets.

The same mechanism produced a 7.00× difference in realised exchange rate between two holders of the same vault with the same `LossUnrealized`: 1.75 drops/share for the sole holder versus 0.25 for the earlier exiter. And it can make a withdrawal **fail** that would otherwise have succeeded — a sole holder's full redemption was refused precisely *because* the waiver inflated her claim from 2,500,000 (exactly the available balance) to 17,500,000 (`1DD536172A22670FCF0526516C5DF0FECAD173EE7CA49695EE8EA58396FB11FC`).

An external reader cannot reproduce any of this without knowing the caller's identity relative to the holder set. So there is no way for a wallet, an indexer, or a risk system to tell a user what their position is worth.

Ask: add a computed `AssetsForWithdrawal` / `ExchangeRate` to the `vault_info` result, calculated the way `VaultWithdraw` calculates it, with an optional `account` parameter so the sole-shareholder waiver can be resolved.

Failing that, document the waiver. It is not in XLS-65, and we found it by reading `VaultWithdraw.cpp` — but it is deliberate, amendment-gated, and spreading. `XRPLF/rippled#8119` "fix: Waive unrealized-loss discount on sole-holder `VaultClawback`" (merged 2026-08-27) explains why it exists: without it, a sole holder's discounted-rate exit converts to 100% of outstanding shares and trips `ValidVault` with `tecINVARIANT_FAILED`. That PR extends the same waiver to `VaultClawback` under `fixCleanup3_4_0`, noting that `VaultWithdraw` has had it under `fixCleanup3_2_0` all along. We have no quarrel with the rule. The problem is that it is now a valuation rule on **two** transactors, keyed to the caller's position in the holder set, with still no read API from which a wallet or a risk system can reproduce it — so the number a user is shown and the number they receive can differ by 7×, as ours did.

M3 — no enumeration and no event stream for vaults, brokers or loans

P1 · rippled / Clio · NEW

To monitor vaults you must already know their IDs. Every route to discovering them is closed:

- `vault_list` is fully specified in XLS-65 §4.2 and returns `unknownCmd` on both servers (D6).
- `loan_info` and `loan_broker_info` also return `unknownCmd`.
- `ledger_data` with a `type` filter is clamped to 256 entries **with the filter applied after the page is

cut**, so a filtered query can legitimately return zero rows on a ledger that contains matches.

- Devnet prunes old ledgers, so back-filling from transaction history is not general. (Corrected

2026-10-13: the submitted text said roughly 29 days. The window is not a fixed number of days; on 2026-10-13, a month after the event, one public Devnet server reported its oldest ledger as 5193077 (closed 10 September); the floor moves and differs between servers.)

- There is no `subscribe` stream for the lending ledger entry types.

The only correct reader we could build is a poller over a hard-coded list of `VaultID`s, which is what we shipped (4-second interval, re-reading each SLE). That is fine for a demo and wrong for an ecosystem: it means no third party can build a vault explorer, a risk dashboard, or a market-wide analytics product without first solving discovery out of band.

Ask, cheapest first: implement `vault_list` (it is already specified, including pagination and three worked response examples); add `loan_info` / `loan_broker_info`; or add the three entry types to `subscribe`. Any one of the three unblocks the category.

M5 — a LoanBroker's action history cannot be reconstructed by filtering, and an indexer cannot tell that it is missing events

P1 · rippled · NEW — found while building, and it shipped a wrong answer before we caught it

To assess a broker you need what they did: when they flagged an exposure, when they declared a loss, when they added or withdrew first-loss capital. The obvious reconstruction is to walk `account_tx` for the broker owner and keep the transactions that concern this broker. There are exactly two ways to decide that, and on an impairment both of them fail.

1. The transaction does not name the broker. `LoanManage` carries `LoanID`. It has no `LoanBrokerID` field at all, on any flag.

2. The broker object is not modified. We expected impairment to touch `LoanBroker` — it is the object whose loss-absorbing obligation the impairment speaks to. It does not. The full affected-node set of `9CFF971F35A9D162CE3A54C71961E64694A83A79321EB05DED8A36EEFAD14106`, an impairment on a funded closed-ended vault with a broker and cover in place, is:

NODE	TYPE
`ModifiedNode`	`Loan`
`ModifiedNode`	`AccountRoot`
`ModifiedNode`	`Vault`

So a filter on either the transaction field or the broker node returns cover deposits and defaults, and **silently drops every impairment**. Defaults survive only because a default decrements `DebtTotal` and therefore does touch `LoanBroker`. The broker's record comes back looking complete, in the right order, with correct figures — and missing precisely the events that show the manager behaving well.

WHAT IT COST US, AND WHY THE SHAPE OF THE FAILURE IS THE POINT

Our conduct score counts impairments to decide whether a loss was *signalled* before it was realised. A manager who flags an exposure and then declares it has warned their depositors; one who goes straight to a default has not. With impairments invisible, every default looked unsignalled. **The rule written to reward disclosure was penalising it**, twenty points per loss, on exactly the managers it was built to protect.

Nothing failed. No result code, no exception, no empty response. The reconstruction returned a plausible history that a reviewer would accept, and we only found it because we happened to bake a vault whose sole action was an impairment and noticed the record said one event where we had submitted two.

That is the part worth fixing. A missing capability announces itself; a filter that quietly under-reports does not, and every consumer of it is wrong in the same direction — too kind to managers who conceal, too harsh on managers who disclose.

THE WORKAROUND, AND WHY IT SHOULD NOT BE NEEDED

The join is available at no extra request: the `Loan` node in the same metadata carries `FinalFields.LoanBrokerID`. So the reconstruction becomes *"keep this transaction if the broker node matches, or the transaction names the broker, or any Loan node in the metadata points at it"*. Sixteen characters of condition, once you know it is required.

Nothing points a reader there. `LoanManage` is documented in terms of the loan, the affected-node set is not published anywhere, and the natural mental model — that an action against a broker's book touches the broker's object — is wrong in exactly one case out of four.

Suggested fix, cheapest first:

1. **Add `LoanBrokerID` to `LoanManage`.** It is knowable at submission, it is already on the `Loan`, and it

makes the transaction self-describing. This alone closes the finding.

1. **Or touch `LoanBroker` on impairment.** There is a defensible reason to: an impairment is a statement

about an obligation that broker carries, and a field such as an impaired-principal counter would make the object tell the truth about its own book. This is the larger change and also the more useful one, because it makes the current state readable without replaying history.

1. **Failing both, document the affected-node set per flag.** A table of what `tfLoanImpair`,

`tfLoanUnimpair` and `tfLoanDefault` each modify would let an indexer author get this right by reading rather than by discovering.

This compounds with M3. There is no event stream and no enumeration, so transaction history is already the only route to a broker's record; that route having a silent hole in it removes the last one.

M4 — XLS-66 states the cover-liquidation formula twice and the two statements disagree

P2 · XLS-66, rippled read APIs · PARTIALLY TRACKED: `XRPLF/XRPL-Standards#623`**, closed without merging**

The cover-liquidation formula **is** specified — XLS-66 §3.10.5 and Appendix A-2 equation (35) — and our wire measurements reproduce it exactly, including that the first factor is the broker's **total debt** rather than the defaulting loan's principal. Our first draft asked Ripple to write down a formula that is already equation (35) of their own standard; we report it now because of what the specification *does* with it, not because it is missing.

What we measured:

```
DefaultCovered = ceil( LoanBroker.DebtTotal × CoverRateMinimum/1e5 × CoverRateLiquidation/1e5 ),
                 then clamped by DefaultAmount and by CoverAvailable
```

We separated the total-debt and loan-principal hypotheses three times with live numbers that diverge:

DEFAULT	DEBTTOTAL	PREDICTION FROM DEBTTOTAL	PREDICTION FROM LOAN PRINCIPAL	ACTUAL COVER CONSUMED	HASH
#1	13,333,337	133,334	100,000	**133,334**	`6F08AF1C308BC429F0726AAB0FBEC8D2AEB8E7B4E867B4A75396953BBA0841B3`
#3	3,666,670	36,667	20,000	**36,667**	`AAC911BD0BF7DC9E0E738B6C56F04E19D490D4D401F5D1039A3C50FCE7401FCB`
#5 (never impaired)	3,666,670	36,667	20,000	**36,667**	`A7DC6A89CC0AE456679473AF5F6431DADE11144D2885303D192F8FD682636085`

Two consequences that a depositor cannot see today:

- The rates **multiply**. `CoverRateMinimum: 10000` and `CoverRateLiquidation: 10000` — "10% cover" —

means at most **1%** of the broker's debt can be consumed by any single default. On a 10 XRP default against 5 XRP of posted cover, 133,334 drops were liquidated and the vault absorbed 9,866,666. Cover absorbed 1.3% of the loss.

- Because the first factor is total debt, **the protection per loan depends on how much other debt the

broker happens to have outstanding**, and total cover paid across a fixed set of defaults depends on the order in which they are declared. `LoanManage` is signed by the broker owner.

Ask 1 — resolve the internal inconsistency. XLS-66 §3.10.5 computes `DefaultCovered = min(MinimumCover × CoverRateLiquidation, DefaultAmount, LoanBroker.CoverAvailable)` — a **three**-term `min`. Appendix A-2 equation (35), line 1944 of the same file on `master`, is a **two**-term `min` with no `CoverAvailable` clamp. An implementer who starts from the appendix can compute a cover deduction larger than the cover that exists. `XRPLF/XRPL-Standards#623` "docs(XLS-66): align fee redirection and `DefaultCovered` with the normative text" was opened on 2026-09-08 to align exactly these two passages, and was **closed without merging** on the same day; we could not find a stated reason. All four of our live defaults clamp on the first term, so we cannot separate the two readings empirically — which is precisely the point.

Ask 2 — say the product out loud, next to the fields. Equation (35) is correct, and almost nobody will read "10% cover" as "1% of debt is consumable". XLS-66 §3.1.11 does work the product numerically (10.9 Tokens), which helps whoever reaches §3.1.11; one sentence in the `LoanBrokerSet` field table, where `CoverRateMinimum` and `CoverRateLiquidation` are actually defined, would reach everyone else.

Ask 3 — expose it. No read API previews the liquidatable amount. A depositor evaluating a broker sees `CoverAvailable`, `CoverRateMinimum` and `CoverRateLiquidation`, and has to reimplement a two-factor product and a three-way clamp before they can tell what "cover" is worth.

---

9. TUTORIALS AND DOCUMENTATION

D2 — the lending tutorial chain does not complete, and dies as a `TypeError` instead of reporting a result code

P0 · `XRPLF/xrpl-dev-portal` · **NEW**

Six tutorial pages run the same setup script: `docs/tutorials/defi/lending/use-the-lending-protocol/{create-a-loan-broker, create-a-loan, manage-a-loan, pay-off-a-loan, deposit-and-withdraw-cover, claw-back-cover}.md`, each via `_code-samples/lending-protocol/js/lendingSetup.js`.

Exact failing step: `Setting up tutorial: 5/7`.

`lendingSetup.js` lines 201–209 create an **open-ended** vault — there is no `VaultKind`, `SubscriptionDate` or `RedemptionDate` anywhere in the file:

```
client.submitAndWait({
  TransactionType: 'VaultCreate',
  Account: loanBroker.address,
  Asset: { mpt_issuance_id: mptID },
  Flags: xrpl.VaultCreateFlags.tfVaultPrivate,
  DomainID: domainID
}, { wallet: loanBroker, autofill: true }),
```

Lines 242–246 then attach a broker to it:

```
client.submitAndWait({
  TransactionType: 'LoanBrokerSet',
  Account: loanBroker.address,
  VaultID: vaultID
}, { wallet: loanBroker, autofill: true }),
```

Under `LendingProtocolV1_1` that returns `tecNO_PERMISSION` (D1.c) — and `submitAndWait` does **not** throw on a `tec`. So execution continues to lines 258–260:

```
const loanBrokerID = loanBrokerSetResponse.result.meta.AffectedNodes.find(node =>
  node.CreatedNode?.LedgerEntryType === 'LoanBroker'
).CreatedNode.LedgerIndex
```

`.find()` returns `undefined`, and the script dies with `TypeError: Cannot read properties of undefined (reading 'CreatedNode')` while the console still reads `Setting up tutorial: 5/7`. **The developer is shown a JavaScript crash, not a ledger result code.** Nothing in that output points at the vault kind, at `LendingProtocolV1_1`, or at the ledger at all. This is the single worst first-hour experience in the stack and it is what a hackathon participant hits at minute ten.

Second, independent break. `_code-samples/lending-protocol/js/package.json` pins:

```
{ "dependencies": { "xrpl": "^4.6.0" } }
```

`^4.6.0` resolves to `4.6.0`, whose `Wallet/utils.js` `computeSignature(tx, privateKey, signAs)` has no signing-role parameter and unconditionally calls `encodeForSigning` — prefix `0x53545800`. With `fixCleanup3_4_0` enabled, rippled expects the counterparty prefix `0x43505400` and rejects the blob locally: `fails local checks: Counterparty: Invalid signature.` `xrpl@5.2.0` adds the `SIGNING_ENCODERS` table that routes counterparty signing through `encodeForSigningCounterparty`. Line 289, `xrpl.signLoanSetByCounterparty(borrower, loanBrokerSignedTx)`, therefore cannot succeed on the pinned version (L2). We proved this mechanism from the shipped `dist` source rather than by re-running the live rejection, and say so because the first break above *was* reproduced live and the two deserve different labels. (`fixCleanup3_4_0` is a rippled amendment, not a library release, so an npm version cannot "predate" it — an earlier draft of this paragraph said exactly that, and a maintainer would have caught it.)

Note the asymmetry, because it affects how you fix this: the tutorial *pages* say `npm install xrpl` (`create-a-loan-broker.md` line 48, `create-a-loan.md` line 51), which since 2026-09-11 22:20 UTC resolves to 5.2.0 and works. The *code-sample folder* pins `^4.6.0` and does not. A reader following the page text and a reader running `npm install` inside the sample folder now get different outcomes.

Suggested fix (we are opening a PR with exactly this):

1. Compute the dates before the vault is created, and explain why in a comment:

``js // LendingProtocolV1_1: a LoanBroker can only be attached to a close-ended vault. // SubscriptionDate must be in the future when the transaction applies (else tecEXPIRED); // RedemptionDate must be at least MIN_INVESTMENT_PERIOD (180 s) after it, and must outlast // every loan's final payment. const RIPPLE_EPOCH = 946684800 const subscriptionDate = Math.floor(Date.now() / 1000) - RIPPLE_EPOCH + 60 const redemptionDate = subscriptionDate + 60 * 60 * 24 * 60 // 60 days; the loans below run 30 ``

1. Add `VaultKind: 1`, `SubscriptionDate`, `RedemptionDate` to the `VaultCreate`.
2. Check `result.meta.TransactionResult` before indexing `AffectedNodes` — on both the `VaultCreate` and the

`LoanBrokerSet`. A tutorial that reports `tecNO_PERMISSION` teaches something; a `TypeError` teaches nothing.

1. Poll the validated ledger's `close_time` past `subscriptionDate` before the `LoanSet` block, because

`LoanSet` returns `tecTOO_SOON` until then and the local clock is not authoritative (U9).

1. Bump `package.json` to `"xrpl": "^5.2.0"`.

Composition verified. `VaultKind: 1` composes cleanly with `tfVaultPrivate`, a `DomainID`-backed Permissioned Domain and an MPT asset — the exact combination the tutorial uses, and the one thing we had listed in §11.1 as untested. We ran the corrected tutorial end to end on Devnet against rippled 3.4.0-rc5 at 2026-09-12 07:40 UTC: vault `495C9ED4E0ECA55942AB0136D0DD46B6A45218737A02391FC80190FFC45741F4` (`VaultKind: 1`, `SubscriptionDate: 842514127`, `RedemptionDate: 847698127`), broker `00BD46B4A49D1C5051C9FBE5C444CA880B20D89A0EB285A048C30776EDFD1EB3`, two loans originated, the whole chain `tesSUCCESS` in 79.6 s. The diff applies cleanly to a fresh checkout of `master`; scripts and run logs are in `reference/pr-tutorial/`.

The Python sample has the identical defect and cannot be fixed from that repository. `_code-samples/lending-protocol/py/lending_setup.py` L270–275 builds the same open-ended `VaultCreate`; `vault_kind`, `subscription_date` and `redemption_date` appear zero times in the file. `requirements.txt` pins `xrpl-py>=4.5.0`, the latest release is 5.1.0, and `xrpl/models/transactions/vault_create.py` on `main` has no `vault_kind` field. Support is in flight in `XRPLF/xrpl-py#1034`. Our PR is JavaScript-only and says so; the Python path is blocked upstream (L3), which is worth a line on the Python tutorial tab.

D3 — three `tec` outcomes that gate the lifecycle are absent from XLS-65, and the docs fix for them is unmerged

P1 · XLS-65, `xrpl-dev-portal` **· PARTIALLY TRACKED:** `xrpl-dev-portal#3923`

We hit these outcomes on the wire and could not find any of them on xrpl.org or in XLS-65. Re-checking the open PR sets on 2026-09-12 changed the shape of this finding twice, so here is where each one actually stands.

TRANSACTION	OUTCOME	CONDITION	WHERE IT IS WRITTEN DOWN NOW	EVIDENCE
`VaultCreate`	`tecEXPIRED`	`SubscriptionDate` is in the past when the transaction applies	`#3923` (`vaultcreate.md`); **not in XLS-65**	`54C94DAADABDFEC8BEC88DD32A8E0A6135E4F5396B2BBFA34733E22769509087`
`LoanSet`	`tecTOO_SOON`	submitted before `close_time` passes `SubscriptionDate`	`#3923` (`loanset.md`); **not in XLS-66**	`6BDF5FC8710F025FB56C47C07B05AB576AE9BFA6CDD65AB09C70B62C7045343A`
`VaultWithdraw`	`tecTOO_SOON`	**the entire Investment phase**, regardless of liquidity	`#3923` (`vaultwithdraw.md`); **not in XLS-65**	`C2799D829A6BC780A53782D57E07642190946C186D0795BE3F437646DD3B6493`

XLS-65 contains zero occurrences of `tecTOO_SOON` and zero of `tecEXPIRED` on `master`, so all of the vault-side phase failures are absent from the standard that defines vaults; and XLS-66's only two `tecTOO_SOON` entries are `LoanManage`'s, so `LoanSet`'s is missing there too. On the xrpl.org side, the open PR `xrpl-dev-portal#3923` adds all three — `vaultwithdraw.md`'s new row reads "`tecTOO_SOON` | The vault is closed-ended and in its _Investment_ phase", which is exactly how we would have asked for it. It is unmerged and targets `release-3.4.0`, so the live pages are still silent. Our contribution is independent confirmation against rc5 on 2026-09-12 that each rule in that diff behaves as written (`BC42B4AB43658F65FD98A9BFF6A9509E340266D260F4D11036031CB7BD27BCB6` for the `VaultWithdraw` case), and a request to prioritise the merge and to mirror the three into XLS-65.

Credit where it is due, because we expected to be reporting two more and were wrong. XLS-66 *does* document the two adjacent cases: impair-too-soon at §3.10.4.2 item 9 (*"`fixCleanup3_4_0`: `tfLoanImpair` is specified and `currentTime <= Loan.NextPaymentDueDate` (can only impair a loan whose payment is already overdue)"*) and late-payment-without-`tfLoanLatePayment` at §3.11.4.2 item 11, both correctly gated on the amendment and both more precise than our own prose was. We hit both on the wire — `E610F5DEE85875C62B3352A1A4A56608E8059AD98369619CF650C8D5B882110F` for the impair case, and three burned transactions plus a 60-second demo window for the `LoanPay` one (`99930F38…`, `2A1D01F2…`, `3AC23CA0…`) — and the reason we hit them is that the *reference pages* do not carry what the standard says: `loanmanage.md` on `master` documents `tecTOO_SOON` only for **defaulting**, `loanpay.md` has no `tecEXPIRED` row at all, and neither file is touched by #3923. That is a smaller and more actionable ask than the one we started with: the spec is right, the pages need to catch up with it.

The `VaultWithdraw` row deserves emphasis because it is counter-intuitive and it is the fact our entire product is built on. `VaultWithdraw` during the Investment phase returns `tecTOO_SOON` even on a vault with **no LoanBroker attached at all** and `AssetsAvailable == AssetsTotal`. It is a phase check, not a liquidity check. Confirmed three ways, including on a fully-liquid vault with no loans and on a deliberately illiquid one (`7CFB311E608FED6C2BEA856821B4E36346870C825ED0616E2692C7BB16538655`, `699B5E89449C93876B166944327AA8D59EC98294D7342F7D06C690719C9485EA`). A UI that treats `canWithdraw: false` as an error state is wrong; it is the protocol working as designed.

D4 — the faucets list still advertises a Lending Devnet whose hostnames do not resolve

P1 · `XRPLF/xrpl-dev-portal` · **NEW** · **one-line fix**

`resources/dev-tools/faucets.json`, on `master`, verified 2026-09-12, still contains:

```
{
  "id": "faucet-select-lending-devnet",
  "wsUrl": "wss://lend.devnet.rippletest.net:51233/",
  "jsonRpcUrl": "https://lend.devnet.rippletest.net:51234/",
  "faucetHost": "lend-faucet.devnet.rippletest.net",
  "shortName": "Lending-Devnet",
  "desc": "Preview of XLS-66d Lending Protocol."
}
```

All three hostnames are **NXDOMAIN**, re-verified today:

```
$ nslookup lend.devnet.rippletest.net         ->  Non-existent domain
$ nslookup lend-faucet.devnet.rippletest.net  ->  Non-existent domain
$ curl https://lend.devnet.rippletest.net:51234/   ->  exit code 6 (could not resolve host)
```

This is the first entry a developer looking for "the lending network" will click, and it is the worst possible first step: a dead host with no error explaining why. Lending now lives on the main Devnet, where it works.

Suggested fix: delete the object. Optionally update `faucet-select-devnet`'s `desc` to mention that `SingleAssetVault` and `LendingProtocol` are enabled there.

D5 — `vault_info`'s documented request parameter name is wrong in XLS-65

P1 · XLS-65 §3.9.1 · NEW

XLS-65 §3.9.1 specifies a single required request field named `vault`. Re-verified against rippled 3.4.0-rc5, 2026-09-12:

```
{"method":"vault_info","params":[{"vault":"6717B511…"}]}
-> {"error":"invalidParams","error_code":31,
    "error_message":"Must specify either 'vault_id' or both 'owner' and 'seq'."}

{"method":"vault_info","params":[{"vault_id":"6717B511…"}]}
-> {"status":"success","vault":{ … }}
```

The xrpl.org `vault_info` page is correct (`vault_id`, or `owner` + `seq`); it is the XLS that is wrong. The server's error message is good and told us the answer immediately — credit where due.

Related, and worth resolving while someone is in that file: xrpl.org documents `ledger_entry`'s `vault` selector as a String only, while `xrpl-py` also accepts an `{owner, seq}` object. One of the two is wrong.

D6 — `vault_list` is fully specified with three worked examples, and unimplemented on both servers

P1 · XLS-65 §4.2 · NEW

§4.2 "RPC `vault_list` (Clio-only)" runs from §4.2.1 to §4.2.9 and includes three complete worked request/response pairs, for MPT, IOU and XRP assets. Re-verified 2026-09-12:

```
rippled:  {"method":"vault_list", …} -> {"error":"unknownCmd","error_code":32,"error_message":"Unknown method."}
Clio:     {"method":"vault_list", …} -> {"error":"unknownCmd","error_code":32,"error_message":"Unknown method."}
```

`vault_info` is implemented on both, so this is specifically `vault_list`. As written the section reads as an available API, and it is the natural answer to "how do I find vaults" — which, per M3, currently has no answer at all.

Suggested fix: implement it, or mark §4.2 `Not yet implemented` with a one-line note. The second option costs a minute and prevents an hour.

D7 — the canonical ledger-entry short-names table omits `Vault`, `Loan` and `LoanBroker`

P1 · `XRPLF/xrpl-dev-portal` · **NEW**

`docs/references/http-websocket-apis/api-conventions/ledger-entry-short-names.md` is the table that the `account_objects` page defers to for valid `type` values ("accepts canonical names of ledger entry types (case insensitive) or short names"). Verified 2026-09-12: 3,805 bytes, **28 data rows** running from `AccountRoot`/`account` to `XChainOwnedCreateAccountClaimID`, and **zero** matches for `loan` or

`vault`.

The servers accept all three, and `xrpl-py`'s `AccountObjectType` already has `VAULT`, `LOAN` and `LOAN_BROKER`. So the values exist everywhere except in the document that is supposed to enumerate them, which means the documented way to list a broker's loans is undiscoverable from the docs.

D8 — XLS-66 gives `LoanPay` transaction type 83; it is 84

P2 · XLS-66 §3.11.1, README line 1347 · NEW · one-character fix

Verified on `master` 2026-09-12. Every other number in both specifications is correct — we checked all of them: `VaultCreate` 65 through `VaultClawback` 70, `LoanBrokerSet` 74, `LoanBrokerDelete` 75, `LoanBrokerCoverDeposit` 76, `LoanBrokerCoverWithdraw` 77, `LoanBrokerCoverClawback` 78, `LoanSet` 80, `LoanDelete` 81, `LoanManage` 82. Only `LoanPay` is wrong. Live `server_definitions`, `ripple-binary-codec` 2.11.0 and `xrpl-py`'s `definitions.json` all say **84**; 79 and 83 are unassigned.

D9 — the draft and the implementation disagree about the loan-maturity buffer, and the draft is wrong

P1 · `XRPLF/XRPL-Standards#587` **§7 · NEW · originally raised as a question; settled by experiment on 2026-09-12**

(Promoted from P2. A draft standard that states a gate on the protocol's central transaction incorrectly costs an implementer hours of unexplained `tecNO_PERMISSION`*s, which is the P1 definition, and it is far cheaper to fix before the PR merges than after.)*

PR #587 specifies: *"If* `startDate + (paymentInterval × paymentTotal)` *is not strictly before* `RedemptionDate`*, return* `tecNO_PERMISSION`*"*, and reiterates that no closed-ended `LoanSet` succeeds unless the final scheduled payment is strictly before `RedemptionDate`. `LoanSet.cpp` requires an additional 60-second `kLoanRedemptionBuffer`.

We originally flagged this as unbracketed, because our own data was consistent with both readings. We then ran the separating experiment against rippled 3.4.0-rc5: 20 `LoanSet` submissions across two fresh close-ended vaults, varying `PaymentInterval` against a fixed `RedemptionDate` and reading the true slack `k = RedemptionDate − (StartDate + PaymentInterval × PaymentTotal)` back off each validated result rather than predicting it.

- **Rejected** with `tecNO_PERMISSION` at k = 47, 51, 53, 56, 57, 59, 62 and 68 s.
- **Accepted** at k = 66, 72, 80, 83 and 198 s.

The draft's rule as written is refuted: k = 57 and k = 59 are strictly before `RedemptionDate` and are rejected (`2F9B898DE63BDDA3C4B57AAE97EC31217A8DC09223BED9D1386265D606250521`, `66A3CC2C3331E9CBFCEF0E63BC314F1B2FFFCBECA17A95133776AA43FCDBD618`). A buffer of roughly 60 s exists, consistent with `kLoanRedemptionBuffer`. We stop short of asserting the constant is exactly 60, because the transactor evaluates against the parent ledger's close time while our `StartDate` proxy is the applying ledger's, and Devnet's `close_time_resolution` is 10 s — the same order as the effect, which is why one probe at k = 68 was rejected while one at k = 66 succeeded (`87692E38C46EA2D16CD2EB8723DFCC895B5EBC47BA7396E263320A4A5F6FC99A`).

Ripple's own documentation already states the rule correctly. The open docs PR `xrpl-dev-portal#3923` writes on `loanset.md`: *"only if the loan's final scheduled payment is at least 60 seconds before the vault enters its Redemption phase"*, with a matching `tecNO_PERMISSION` row and the observation that the maximum term of new loans shrinks as the vault approaches its `RedemptionDate`. So the docs team has it and the standard does not.

Ask: one clause in PR #587 §7 before it merges — "strictly before" → "at least `kLoanRedemptionBuffer` (60 s) before" — and a statement of the constant by whoever owns `LoanSet.cpp`.

D10 — twelve Invariants sections, plus XLS-65's Rationale and Security Considerations, are `TBD`

P2 · XLS-65, XLS-66 · PARTIALLY TRACKED: `XRPLF/XRPL-Standards#616` **"Add loan invariants" (open since 2026-09-02) and** `#555` **"XLS-65: Add global Vault ledger entry invariants (3.1.10)" (open since 2026-05-27)**

Verified on `master`, 2026-09-12:

DOCUMENT	TBD SECTIONS
XLS-65	§3.1.10 Invariants, §3.2.5.1 Data Verification, §3.2.7 Invariants, §3.7.4 Invariants, §3.9.3 Failure Conditions, §3.9.4 Example Request, **§4 Rationale**, **§5 Security Considerations**
XLS-66	§3.1.8, §3.2.7, §3.3.5, §3.5.5, §3.6.5, §3.7.5, §3.8.7, §3.10.6, §3.11.6 — all Invariants

Twelve of those are Invariants. XLS-65 §4 and §5 are literally `_TBD_`. XLS-66 §5 does exist and is one paragraph, in full:

> *The protocol makes strong trust assumptions between Vault Depositors, LoanBrokers, and Borrowers. The protocol does not offer on-chain algorithmic protection against default, thus all protocol participants must perform their due diligence and necessary off-chain checks.*

That paragraph is honest and correct as far as it goes. But the invariants are the part an integrator most needs, because they are the promises they are allowed to rely on — and for most of the vault surface they do exist in C++ even where the spec says `TBD`. **For the lending flow specifically they exist nowhere.** `XRPLF/rippled#7690` "Vault value-conservation invariant is a no-op for the entire lending flow", open since 2026-07-01, reports that in `VaultInvariant.cpp` the `ttLOAN_SET`, `ttLOAN_MANAGE` and `ttLOAN_PAY` branches are literally `// TBD -> return true`, that the only conservation checks on that path are `#if !NDEBUG` asserts which compile out in Release, and that nothing ties the vault's `AssetsAvailable` to its pseudo-account's real balance the way `LoanBrokerInvariant` does. So the three transactions that move the most value in this protocol are `TBD` in the specification **and** `TBD` in the enforcement. We are not re-reporting #7690; we are pointing out that it and the twelve empty Invariants sections are the same gap seen from two sides, and that quoting them together is the strongest argument either one has.

Two PRs are filling some of this in and both are unmerged: `#616` "Add loan invariants" (+83/−3 across two files, opened 2026-09-02) and `#555` "XLS-65: Add global Vault ledger entry invariants (3.1.10)" (+373/−94, opened 2026-05-27, last updated 2026-09-11) — the latter targeting the first row of the table above. Between them they would close a real share of the twelve. Neither touches XLS-65 §4 Rationale or §5 Security Considerations, which remain `_TBD_`.

We raise it with a specific context rather than as a general complaint: Ripple, Clearpool and Cicada announced an RLUSD institutional lending platform built on exactly these two amendments on 2026-08-20/21, currently testing on XRPL Devnet. For a protocol that is weeks away from pooling third-party institutional capital, an empty Security Considerations section and twelve empty Invariants sections are the gap most likely to be quoted back at the project later.

D11 — the specifications state the arithmetic and never once evaluate it

P1 · XLS-65, XLS-66, xrpl-dev-portal · NEW

Neither standard contains a worked numeric example. Not a NAV, not a share price, not a cover liquidation, not an interest accrual. Every rule is given as an expression over named fields, and no expression is ever evaluated against values.

This is not a stylistic complaint and it is not about readability. It has a specific consequence, which we can demonstrate on ourselves.

What it cost us. XLS-66 gives cover liquidation as

```
min( DebtTotal x CoverRateMinimum x CoverRateLiquidation, DefaultAmount, CoverAvailable )
```

Three things about that line are invisible until you put numbers into it:

1. **Both** rates are in 1e-5 units, so **both** are divided by 1e5. Nothing on the

page says so — the only written-down source is a constant inside `node_modules` (U3). Read naively, 10% reads as `10000` and the product is wrong by 1e10.

1. The result is **ceilinged**, which the prose does not mention. M4 records that the

standard's own two statements of this formula disagree with each other.

1. The base is `LoanBroker.DebtTotal` — the broker's **total** book at that instant —

not the principal of the loan that defaulted.

We lost roughly two hours across (1) and (2). One line reading *"a broker with 40 XRP outstanding,* `CoverRateMinimum` *10000 and* `CoverRateLiquidation` *10000, pays* `ceil(40 × 0.1 × 0.1)` *= 0.40 XRP"* would have cost one sentence and closed all three questions at once.

The consequence that matters. Point (3) is not a rounding detail. Because the base is the total book, and the book shrinks with each default, **the total cover paid across a fixed set of defaults depends on the order they are declared in** — and the party who chooses that order is the party whose capital is being consumed. Measured on Devnet with loans of 30 and 10 XRP at 10%/10%: declaring the big one first consumed 0.50 XRP of cover, the small one first consumed 0.70 XRP. Same losses, same rates, 40% difference in what the first-loss capital absorbed. The difference lands on depositors.

That behaviour is fully determined by the specification as written. Discovering it required no undocumented rule and no source reading — only substituting two numbers. As far as we can establish nobody had evaluated it, because nothing in the document invites you to put a number in and look at what comes out. An unevaluated rule reads as a rule with no consequences.

An XRPL engineer who worked on the lending design told us, unprompted, that a business example with real numbers would have saved him half an hour of explaining the concept to us in person. We are filing it because by the time he said it we had already paid for the same gap twice, in a form we could measure.

Suggested fix, in the order we would do it:

1. **One worked example per formula**, inline, in the section that states it. Cover

liquidation, share price on deposit, share price on withdrawal, interest accrual, management fee. Five examples, five lines each. This is an afternoon.

1. **One end-to-end worked lifecycle** on the docs portal: a vault takes 50 XRP, lends

40 across two loans, one defaults, and every intermediate figure is shown. This is the artefact that would have replaced our first day.

1. **State units at the field**, not in a constant. `CoverRateMinimum` should read

"1e-5 units; 10000 = 10%" where it is defined.

---

10. ITEMS WE WITHDREW AFTER RE-VERIFICATION

We include these because a report you cannot audit is worth nothing, and because two of them were in our own internal notes as findings until this morning.

W1 — "**XLS-65 gives the vault transaction type numbers as 58–63.**" Recorded during reconnaissance. Re-verified against `master` on 2026-09-12: the numbers are correct (`VaultCreate` 65 through `VaultClawback` 70). Already fixed upstream. **Withdrawn.**

W2 — "**The** `ledger_entry` **method page does not document** `vault`**,** `loan` **or** `loan_broker`**.**" Re-verified: the page documents all three with dedicated sections, including `loan`'s `loan_broker_id` + `loan_seq` and `loan_broker`'s `owner` + `seq`. **Withdrawn.** Note this is a *different* page from D7: the short-names table, which `account_objects` defers to, does still omit them, and that finding stands.

W3 — "`LoanSet` **requires** `Account` **to be the borrower and** `Counterparty` **to be the broker owner.**" One of our probes observed `tecNO_PERMISSION` with the broker as `Account` and concluded the roles were reversed. Two other probes succeeded with the broker as `Account`, and `create-a-loan.md` line 141 states that the fields can be swapped and only determine signing order. The `tecNO_PERMISSION` was the maturity constraint (U1). **Withdrawn** — and offered as the clearest illustration we have of what an overloaded result code costs: it very nearly turned into a false bug report against a library that was behaving correctly.

---

11. SCOPE, AND WHAT WE DID NOT TEST

11.1 Not tested

Stated so that nobody builds on a claim we did not make.

- **Non-XRP vault assets.** Everything above uses XRP, so `Vault.Scale` is 0 and all arithmetic is in

integer drops. With an IOU or MPT asset and a non-zero scale, the `clampToAssetsTotalScale`, `debitIsNonZeroDust` and `tecPRECISION_LOSS` paths become far more active and several of our exact numbers will shift. **For an RLUSD-denominated deployment this is the first thing we would re-run.**

- **Whether anyone other than the broker owner can submit** `tfLoanImpair`**.** Every impairment we ran was

signed by the broker owner.

- **The** `min()` **clamps in the cover formula.** In four defaults the double product was always far below both

`DefaultAmount` and `CoverAvailable`, so only the first term ever bound.

- **`LoanDelete`, `LoanBrokerDelete`, `VaultDelete`, `LoanBrokerCoverClawback`, `tfLoanFullPayment`,

`LatePaymentFee`, `ClosePaymentFee`, `OverpaymentFee`, `CloseInterestRate`.**

- `OracleSet` **under a loaded ledger or a regular-key account.** Devnet `load_factor` was 1 throughout.

11.2 Prior-art searches, and what they missed

Before marking anything **NEW** we searched `XRPLF` across `rippled`, `clio`, `xrpl.js`, `xrpl-py`, `xrpl-dev-portal` and `XRPL-Standards` for the field names, error strings, transaction names and symptoms in each finding — among them `counterparty signature`, `signLoanSetByCounterparty`, `encode_for_signing_counterparty`, `43505400`, `fixCleanup3_4_0`, `VaultKind`, `closed-ended vault`, `close-ended`, `LoanBrokerSet`, `tecNO_PERMISSION lending`, `LoanSet flags`, `txToFlag`, `tfLoanOverpayment`, `ledger_entry loan`, `vault_list`, `ledger-entry-short-names`, `lend.devnet`, `lending tutorial`, `VaultInfoResponse`, `PreviousFields` and `LossUnrealized` — and read the open PR sets on `XRPLF/XRPL-Standards`, `XRPLF/xrpl.js` and `XRPLF/xrpl-py`.

That pass had two defects, and we would rather state them than have them found. It did not cover `xrpl-dev-portal`'s pull requests, and it read PR titles rather than file-level diffs. We re-ran it on 2026-09-12 across issues *and* pull requests, open *and* closed, including `xrpl-dev-portal` PRs and their diffs. Five findings changed as a result:

FINDING	WHAT THE RE-RUN FOUND	WHAT WE DID
X1	`rippled#6487` describes the identical defect and was self-closed as "Not a bug"	Dropped the NEW marker; the closure is now the finding (§5)
U5	`xrpl-dev-portal#3612` already proposes the documentation fix, scoped to lending entries	Marked already reported; kept only the part #3612 does not cover
U4	`xrpl.js#3435` already covers the `Scale` 11–20 range	Cited it; the layer contradiction is what remains NEW
U10	`XRPL-Standards#497` + `rippled#6528` already specify and implement the `VaultID` fix	Cited both
M1	`xrpl.js#3364` (merged 2026-08-20) shipped XLS-0096 bindings in `xrpl@5.2.0`	Deleted our claim that no SDK exists, and the ask built on it

Reading the `xrpl-dev-portal` diffs also rewrote D1's closing section, D3 and D9, because `xrpl-dev-portal#3923` — whose body is empty and which we had recorded as a placeholder — in fact documents the `LoanBrokerSet` closed-ended gate, three of the lifecycle `tec` outcomes, and the 60-second maturity buffer.

We say all of this plainly because the difference between a report and a complaint is whether you checked, and one of our misses was our own lead finding. A §11.2 you cannot catch us being wrong on is worth nothing.

Where a search found something, we cite it and comment there rather than opening a duplicate — see X1, L5, L8, L9, L11, L13, U3, U4, U5, U10, M4, D1, D3, D9 and D10. Four more findings are themselves new but cite upstream work that already implements the fix they ask for, which we think is a stronger thing to say than asking twice: U1 (`rippled#7848`), U2 (`rippled#7950`), M1 (`XRPL-Standards#611`) and M2 (`rippled#8119`).

Where several good-faith searches found nothing at all, we say so and mark it **NEW**: that applies to the **eighteen** findings D2, L2, L3, L4, L6, L7, L10, L12, U6, U7, U8, U9, M3, D4, D5, D6, D7 and D8, and to the primary claims of U1, U2 and M2. L1 belongs to neither group — it was live when we hit it and was fixed upstream before we could report it (§13).

---

12. IF ONLY THREE THINGS GET FIXED

1. **Merge `xrpl-dev-portal#3923`, merge XRPL-Standards PR #587 and PR #582, and add the `LoanBrokerSet`

closed-ended condition to XLS-66 §3.3.3.2** (D1). All of the material exists and all of it is in review while an ecosystem builds against a network whose only *published* description is wrong. #3923 fixes a developer's first hour; #587, #582 and the one added failure condition fix the record.

1. **Fix the lending tutorial's `VaultCreate` and its result-code check** (D2). Four lines of transaction

fields and one `if`. It is the difference between a developer's first hour ending in a working loan and ending in an unexplained `TypeError`. We have the PR ready and run green against rc5.

1. **Document that `PreviousFields` records nothing for a field that was absent before the transaction**

(X1). One paragraph. Without it, every indexer built on this protocol will silently miss the first impairment of every healthy vault, which is the only credit event that matters — and one explorer operator reported exactly that in March 2026 (`rippled#6487`).

Everything else in this report is smaller than these three.

---

13. VERIFICATION

Last re-tested: 2026-09-12, between 03:00 and 08:30 UTC, during the event window.

Protocol claims	XRPL Devnet, rippled **3.4.0-rc5**, `network_id` 2, 89 amendments including `LendingProtocolV1_1` and `fixCleanup3_4_0`. Every `tec`/`tem` behaviour in this document was re-submitted live, not recalled from the reconnaissance runs.
JavaScript claims	`xrpl` **5.2.0** stable with `ripple-binary-codec` **2.11.0**, read from the installed package rather than from the repository
Python claims	`xrpl-py` **5.1.0** (still PyPI `latest`) and **5.2.0b0**, both installed from PyPI
Specification claims	`XRPLF/XRPL-Standards` `master`, plus the diffs of PRs #497, #555, #570, #582, #587, #611, #616, #623, #625
Documentation claims	`XRPLF/xrpl-dev-portal` `master`, plus the file-level diff of open PR #3923
Prior art	Re-searched across `rippled`, `clio`, `xrpl.js`, `xrpl-py`, `xrpl-dev-portal` and `XRPL-Standards`, issues and pull requests, open and closed (§11.2)

Two items were fixed upstream during the event window itself, and both are reported here as resolved rather than as live bugs:

- **L1** — a close-ended `VaultCreate` could not be serialized by the published stable client for 17 days.

Fixed by `ripple-binary-codec` **2.11.0**, published **2026-09-11 20:56 UTC**, which reaches any fresh install because the codec range floats; `xrpl` **5.2.0** followed at **22:20 UTC**, 8 h 40 m before the hackathon opened. We verified the fix end to end on Devnet with a stock install. Two residual gaps remain and are stated as such — the failure mode is unchanged, and the fix landed in the codec while the model layer still has no `VaultKind`.

- **L2 (JavaScript half)** — counterparty signing used the wrong prefix. Fixed in `xrpl` **5.2.0**

(2026-09-11 22:20 UTC) via the `SIGNING_ENCODERS` role table plus `counterpartyTransactionSig` `0x43505400` in `ripple-binary-codec` 2.11.0; confirmed on the wire with unpatched 5.2.0 on rc5. The **Python** half is not fixed in any published release, and that is what L2 now reports.

We kept both, framed as resolved, for one reason: a developer following the published documentation as recently as last week would have hit them, with no way to tell that the fault was not theirs. The lesson we would draw is not about either bug. It is that the client releases landed **hours** before a Ripple-sponsored hackathon on the amendment they support, and that the specification and the reference documentation for that amendment are still in review behind them. That ordering — network, then client, then documentation, then standard — is the pattern underneath most of §6 and all of §9.

Beyond those two, re-verification changed eight more findings and we have marked each one where it changed:

- **X1, U4, U5, U10** gained prior-art citations and lost or narrowed their **NEW** markers, and **M1** lost

a claim outright — `xrpl@5.2.0` ships the XLS-0096 bindings we had said did not exist.

- **D1, D3 and D9** were rewritten after we read the file-level diff of `xrpl-dev-portal#3923` rather than

its (empty) body. It documents more of this report than we had credited it with.

- **M4** was rewritten after we found the formula we said was missing, in two places in XLS-66 that do not

agree with each other. The finding is now the disagreement.

- **U6** was corrected because it contradicted D2 in this same document. D2 was right.
- **D9** was additionally settled by a 20-transaction experiment on rc5 rather than left as an open question.

No finding in this document failed to reproduce on the 2026-09-12 toolchain. The three that did not reproduce were withdrawn before submission and are in §10, with the reasoning intact.

Prepared for the De Vinci Blockchain XRPL Lending Protocol Hackathon, 12–13 September 2026. All transaction hashes are on XRPL Devnet and are viewable at `https://devnet.xrpl.org/transactions/<hash>`.

Appendix C — Composability evidence (Loaded track)

Assembled Sat 12 Sept 13:40. **Input for the Sunday rewrite of** `FEEDBACK.md` **friction point 3**, which is currently framed around the composition we abandoned (a ZK verifier and the lending amendments living on disjoint devnets) rather than the one we shipped. The workshop explicitly invited feedback on combining amendments and on whether the documentation and limitations are well explained. This is that material.

What we composed, in one working flow: XLS-65 Vault · XLS-66 Lending · MPTokensV1 (vault shares) · XLS-80 Permissioned Domains · XLS-70 Credentials · XLS-47 Price Oracle. Six amendments, four of which had to agree with each other before a single gated deposit could settle.

C1 · The gate is not where you look for it

`DomainID` is **not** a field on the `Vault` ledger entry. It lives on the **share** `MPTokenIssuance`, read with `ledger_entry { mpt_issuance: Vault.ShareMPTID }`. Composing XLS-65 with XLS-80 therefore means knowing that the permission sits on an object the Vault only points at. Nothing in the XLS-65 or XLS-80 text says where to look. *Proposal: one sentence in the XLS-65 private-vault section naming the object that carries* `DomainID`*, and a cross-link from XLS-80.*

C2 · The domain is enforced per value movement, not per onboarding

A graded LP paying vault shares to an ungraded account returns `tecNO_AUTH`, so the secondary-market laundering route is closed — good, and undocumented. But `MPTokenAuthorize` by an ungraded account **succeeds**: anyone may create the empty holding object, they simply cannot receive value into it. The mental model "the domain decides who is in the vault" is wrong; it decides who may *receive*. *Proposal: state the enforcement point explicitly, and that holding creation is deliberately ungated.*

C3 · Two ungated paths into a gated vault

The **vault owner is exempt from their own domain**: an ungraded owner deposited into their own gated vault, `tesSUCCESS`. And `LoanBrokerCoverDeposit` is ungated entirely, because posting first-loss capital is not a share purchase. Both are defensible; neither is written down, and both matter to anyone modelling who can put money into a permissioned vault. *Proposal: an explicit exemptions list in the XLS-65 permissioned-vault section.*

C4 · XLS-75 PermissionDelegation does not compose with the lending suite

Measured with a control. `DelegateSet` granting `Payment` → `tesSUCCESS`. `DelegateSet` granting `LoanManage`, `VaultCreate`, `VaultDeposit`, `LoanBrokerSet`, `LoanSet` or `LoanPay` → `temMALFORMED`. Cause: `TxSettings.h` defaults `delegable{Delegation::NotDelegable}` and no transaction in the 65–85 range opts in; 57 other transaction types do. `PermissionDelegationV1_1` is enabled on Devnet, so the amendment is live and simply does not cover this suite.

Consequence for a real deployment: only the LoanBroker owner may impair, unimpair or default, and that owner is necessarily also the vault owner, so an institution cannot delegate the impairment desk to a risk committee through the documented delegation mechanism. The working substitute is `SetRegularKey` or `SignerListSet` — both verified `tesSUCCESS` for `tfLoanImpair` — because preclaim compares the **account**, not the signing key. That is a key-management answer to what looks like a permissions question. *Proposal: say in XLS-66 that lending transactions are non-delegable and name the RegularKey/SignerList pattern as the supported route;* `temMALFORMED` *gives a developer no hint that delegation is simply out of scope.*

C5 · XLS-47 accepts a non-price payload, and nothing says so

An Oracle object carries our fragility vector with `AssetClass: "risk"` and a 40-hex `BaseAsset` derived from the VaultID, and `get_aggregate_price` then computes median and standard deviation across independent publishers **inside the ledger**. This is the cleanest composition in the project and the docs frame XLS-47 purely as a price feed. Three traps we hit: `OracleSet` is **not a merge** (omitted pairs are kept but have `AssetPrice` and `Scale` stripped); `LastUpdateTime` must **strictly increase** or `tecINVALID_UPDATE_TIME` burns a fee and a sequence number; and a **future** `LastUpdateTime` bricks the object until wall clock catches up. *Proposal: document the non-price use, and put those three behaviours in the* `OracleSet` *reference.*

What to do with this on Sunday

Re-frame friction point 3 to lead with C1–C4 — a composition we completed, with the documentation gaps it exposed — and keep the state-proof ask as a short closing paragraph rather than the headline. "We combined six amendments and here is where the documentation stopped" is the answer to the question the jury actually asked; "we could not combine two amendments that live on different devnets" is not.

Appendix D — Vault share token metadata (the distribution channel)

Measured Sat 12 Sept ~15:00, directly after a Ripple developer advocate suggested using the share token's own metadata to carry a pointer to an off-ledger NAV service, so that valuation travels with the token. Three findings, all reproduced on Devnet, all filed to the capture hook.

D1 · Share metadata is write-once and can never be updated by anyone

P1 · XLS-65 · protocol · NEW

STEP	RESULT
`MPTokenMetadata` set at `VaultCreate`	`tesSUCCESS`, persists on the share `MPTokenIssuance`
`VaultSet` with `MPTokenMetadata`	rejected at deserialization: *"Field 'MPTokenMetadata' found in disallowed location."*
`MPTokenIssuanceSet` by the vault owner	`tecNO_PERMISSION`

The share token's issuer is the **vault pseudo-account**, which carries `lsfDisableMaster` and has no regular key, so it cannot sign an update to its own token. The vault owner is not the issuer, so `MPTokenIssuanceSet` refuses them. There is no third path. The metadata is therefore immutable for the life of the vault.

This does not break the idea, it disciplines it: anything embedded there must be a **stable pointer**, never a value. But it also means a URL placed at creation has **no migration path** if the service ever moves.

Proposal. Admit `MPTokenMetadata` on `VaultSet`, which already authenticates the vault owner and is the natural place for it. Failing that, state in XLS-65 that share metadata is write-once — the reasonable assumption, with `DynamicMPT` enabled on the same network, is that `MPTokenIssuanceSet` will work.

D2 · Every vault share is XLS-89 non-compliant by default, so explorers cannot index it

P2 · XLS-65 / XLS-89 · docs · NEW

`VaultCreate` accepted our metadata and rippled then returned, unprompted:

"MPTokenMetadata is not properly formatted as JSON as per the XLS-89 standard. While adherence to this standard is not mandatory, such non-compliant MPToken's might not be discoverable by Explorers and Indexers in the XRPL ecosystem." — followed by the expected fields: `ticker`, `name`, `icon`, `asset_class`, `issuer_name`.

The warning is genuinely good and we would not have known about XLS-89 without it. The gap is that **nothing upstream points a vault creator at it**: XLS-65 does not mention XLS-89, `MPTokenMetadata` is optional on `VaultCreate`, and a vault created the obvious way therefore ships a share token that explorers will not list.

Proposal. Reference XLS-89 from the XLS-65 `VaultCreate` field table, and give one compliant example blob for a vault share.

D3 · No `asset_class` value describes a yield-bearing vault claim

P2 · XLS-89 · docs · NEW

The permitted values are `rwa`, `memes`, `wrapped`, `gaming`, `defi`, `other`. A vault share is a yield-bearing, redeemable claim on a pool of assets whose value moves with credit performance. `defi` is closest and tells a downstream consumer almost nothing; `other` is worse. Given XLS-65 exists specifically to tokenise pooled claims, the taxonomy has a hole exactly where the new primitive sits.

Proposal. Add a `vault-share` (or `pooled-claim`) class, and consider a reserved optional field for a valuation endpoint, since a redeemable claim has a price that the token itself cannot carry.

Why these three matter together

They are the difference between a vault share being an opaque MPT and being a **self-describing collateral instrument**. A broker who receives one as collateral should be able to read, from the token, what it is and where its honest valuation lives. Today they can read neither.

Appendix E — Phase rejection matrix, and an overloaded result code

Captured Sat 12 Sept on Devnet. Serves two purposes: it is the Track 2 minimum-bar evidence (rejected `VaultDeposit`, `VaultWithdraw` and `LoanSet` at the wrong phase), and it produced the clearest error-clarity finding of the event.

Every rejection below sits next to a **control** that succeeds in the phase where the same transaction is legal. Without controls a rejection proves nothing: it could be a malformed transaction or a funding problem.

PHASE	TRANSACTION	EXPECTED	RESULT	HASH
OpenEnded	VaultDeposit (control)	succeed	`tesSUCCESS`	`9D7E7CFE00A554FA065C30A6...`
OpenEnded	VaultWithdraw (control)	succeed	`tesSUCCESS`	`E394EA9BD518BEE31275EC9E...`
OpenEnded	LoanBrokerSet on an OPEN vault	reject	`tecNO_PERMISSION`	`CE477E29A099A6BBC30EBA52...`
Subscription	VaultDeposit (control)	succeed	`tesSUCCESS`	`202F9600CC1D85266FFB27B7...`
Subscription	VaultWithdraw (control)	succeed	`tesSUCCESS`	`92558A56C122F770B8601217...`
Subscription	LoanSet	reject	`tecTOO_SOON`	`E1121DED12D4A320FB58F77D...`
Investment	VaultDeposit at wrong phase	reject	`tecEXPIRED`	`7A10D8053FEE687A008F17E2...`
Investment	VaultWithdraw at wrong phase	reject	`tecTOO_SOON`	`CC104682C298DFB3619EE4C1...`
Investment	LoanSet	succeed	`tecNO_PERMISSION`	`A926030097C2AD2F8ED4D1D6...`
Redemption	VaultDeposit at wrong phase	reject	`tecEXPIRED`	`DEA67094F92657625C795FD3...`
Redemption	VaultWithdraw (control)	succeed	`tesSUCCESS`	`D26222EC26111FFC4F77D3CC...`
Redemption	LoanSet	reject	`tecEXPIRED`	`8A09A988DD3C5433B30E92F5...`

THE ROW THAT IS NOT A FAILURE

`Investment | LoanSet | succeed | tecNO_PERMISSION` is the twelfth attempt, and it is a finding rather than a broken test. The loan was legal by phase, on a funded closed-ended vault with a broker and cover in place. Its term simply landed inside the redemption buffer.

Isolated by varying one thing, on a vault with a 900 second redemption window:

TERM	WINDOW REMAINING	RESULT
180 s	888 s	`tesSUCCESS`
1200 s	879 s	`tecNO_PERMISSION`

This is the third cell of U1, re-derived independently. We did not set out to confirm U1; we set out to satisfy Track 2's wrong-phase requirement, and the twelfth row refused to behave. U1 reached the same conclusion before the event by varying `PaymentInterval` on a build we no longer run. This run varied the *term* instead, on `3.4.0-rc5`, on a vault built for a different purpose — and landed in the same place. Two different isolations, two builds, one cause.

We are reporting it as confirmation rather than as a new finding because the value is in the second observation, not the first. A single probe hitting an overloaded code can always be a mistake in the probe — which is exactly what happened to us once already, and is recorded as withdrawn in §10. Two independent probes cannot.

So `tecNO_PERMISSION` on `LoanSet` means *this loan would mature after the vault redeems* — the same code `LoanBrokerSet` returns when the vault is *open-ended rather than closed-ended*, and the same code `LoanManage` returns when a loan is already impaired. Two of those are reachable within sixty seconds of each other while originating one loan, and nothing in the result distinguishes them.

This inverts the question the workshop posed. The concern raised there was that a confusing error might be returned where a clean permission error belongs. On the lending path the opposite is true: the permission error is the one that is confusing, because it is carrying four meanings at once.

WHY THIS MATTERS FOR THE REPORT

Three codes, sensibly assigned, and the phase model is coherent once you know it. The problem is that none of it is written down: `tecTOO_SOON` appears zero times in the XLS-65 withdrawal failure conditions in our snapshot, and "investment phase" matches nowhere in either specification. Every developer building an LP-facing interface has to discover the lockup empirically, as we did.

And `tecNO_PERMISSION` carries four meanings across three transactions in the same flow (U1), which is the cheapest thing on this list to fix — `rippled#7848` already computes the text that would disambiguate it and discards it.